

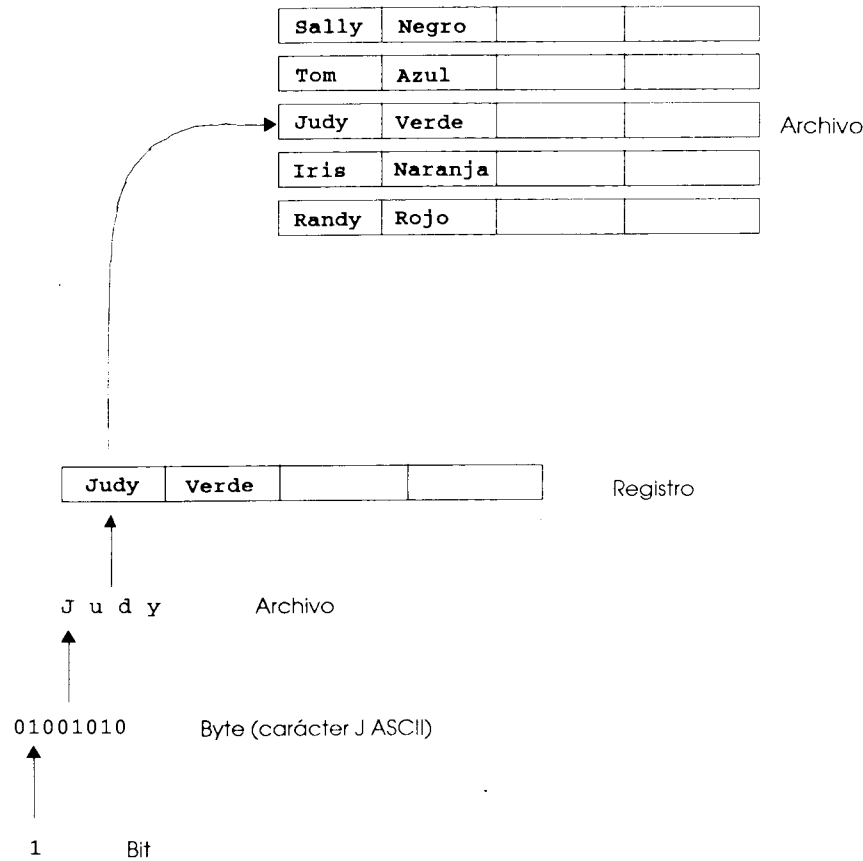


Fig. 11.1 La jerarquía de datos.

registros normalmente se ordenan por número de seguridad social. El primer registro de empleado del archivo contiene el número más bajo de seguridad social, y los registros subsiguientes contienen números de seguridad social cada vez más altos.

Para almacenar datos la mayor parte de los negocios utilizan muchos archivos distintos. Por ejemplo, las empresas pueden tener archivos de nóminas, archivos de cuentas por cobrar (enlistando el dinero que los clientes les deben), archivos de cuentas por pagar (enlistando dinero que se les debe a los proveedores), archivos de inventarios (enlistando hechos relacionados con todos los elementos manejados por el negocio) y muchos otros tipos de archivos. A veces un grupo de archivos relacionados se conoce como una *base de datos*. Una colección de programas diseñado para crear y administrar bases de datos se conoce como un *sistema de administración de bases de datos* (DBMS, por *database management system*).

11.3 Archivos y flujos

C ve cada uno de los archivos simplemente como un flujo secuencial de bytes (figura 11.2). Cada archivo termina con un *marcador de fin de archivo* o en un número de bytes específico registrado en una estructura administrativa de datos, mantenida por el sistema. Cuando un archivo *se abre*, se asocia un flujo con el archivo. Al empezar la ejecución de un programa automáticamente se

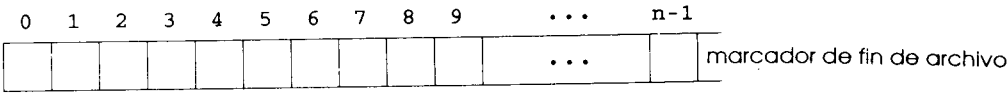


Fig. 11.2 Vista en C de un archivo de n bytes.

abren tres archivos y sus flujos asociados —la *entrada estándar*, la *salida estándar* y el *error estándar*. Los flujos proporcionan canales de comunicación entre archivos y programas. Por ejemplo, el flujo de entrada estándar permite que un programa lea datos del teclado, el flujo de salida estándar permite que un programa imprima datos a la pantalla. Abrir un archivo regresa un apuntador a una estructura **FILE** (definida en `<stdio.h>`) que contiene información utilizada para procesar dicho archivo. Esta estructura incluye un *descriptor de archivo*, es decir un índice a un arreglo del sistema operativo, conocido como una *tabla de archivo abierto*. Cada elemento del arreglo contiene un *bloque de control de archivo* (FCB, por *file control block*) utilizado por el sistema operativo para administrar el archivo particular. La entrada estándar, salida estándar y error estándar son manejados utilizando los apuntadores de archivo `stdin`, `stdout` y `stderr`.

La biblioteca estándar proporciona muchas funciones para leer datos de los archivos y para escribir datos a los archivos. La función `fgetc`, al igual que `getchar`, lee un carácter de un archivo. La función `fgetc` recibe como un argumento un apuntador **FILE** para el archivo del cual se leerá un carácter. La llamada `fgetc(stdin)` lee un carácter de `stdin` —la entrada estándar. Esta llamada es equivalente a la llamada `getchar()`. La función `fputc`, al igual que `putchar`, escribe un carácter a un archivo. La función `fputc` recibe como argumentos un carácter para ser escrito, y un apuntador al archivo hacia el cual el carácter será escrito. La llamada de función `fputc('a', stdout)` escribe el carácter 'a', a `stdout` —la salida estándar. Esta llamada es equivalente a `putchar('a')`.

Varias otras funciones, utilizadas para leer datos de la entrada estándar y para escribir datos a la salida estándar, tienen funciones de procesamiento de archivo similarmente identificados. Las funciones `fgets` y `fputs`, por ejemplo, pueden ser utilizadas para leer una línea de un archivo y para escribir una línea a un archivo, respectivamente. Sus contrapartidas para leer de la estrada estándar y para escribir a la salida estándar, `gets` y `puts`, ya fueron analizadas en el capítulo 8. En varias de las siguientes secciones, presentamos los equivalentes en procesamiento de archivo de las funciones `scanf` y `printf`—`fscanf` y `fprintf`. Más adelante en el capítulo analizaremos las funciones `fread` y `fwrite`.

11.4 Cómo crear un archivo de acceso secuencial

C no impone estructuras a un archivo. Por lo tanto, como parte del lenguaje C no existen conceptos como registro de un archivo. Por lo tanto, para que cumpla con los requisitos de cada aplicación en particular, el programador deberá proporcionar alguna estructura de archivo. En el ejemplo siguiente, vemos como un programador puede imponer una estructura de registro en un archivo.

El programa de la figura 11.3 crea un archivo simple de acceso secuencial, que podría ser utilizado en cualquier sistema de cuentas por cobrar para ayudar a llevar control de las cantidades que deben los clientes a crédito de una empresa. Para cada cliente, el programa obtiene un número de cuenta, el nombre del cliente y el saldo del mismo (es decir, la cantidad que el cliente le debe a la empresa debido a bienes y servicios recibidos en el pasado). Los datos obtenidos de cada cliente constituyen un "registro" para cada uno de estos clientes. En esta aplicación el número de cuenta se utiliza como registro clave —el archivo será creado y mantenido por orden de número

```
/* Create a sequential file */
#include <stdio.h>

main()
{
    int account;
    char name[30];
    float balance;
    FILE *cfPtr; /* cfPtr = clients.dat file pointer */

    if ((cfPtr = fopen("clients.dat", "w")) == NULL)
        printf("File could not be opened\n");
    else {
        printf("Enter the account, name, and balance.\n");
        printf("Enter EOF to end input.\n");
        printf("? ");
        scanf("%d%s%f", &account, name, &balance);

        while (!feof(stdin)) {
            fprintf(cfPtr, "%d %s %.2f\n",
                account, name, balance);
            printf("? ");
            scanf("%d%s%f", &account, name, &balance);
        }

        fclose(cfPtr);
    }

    return 0;
}
```

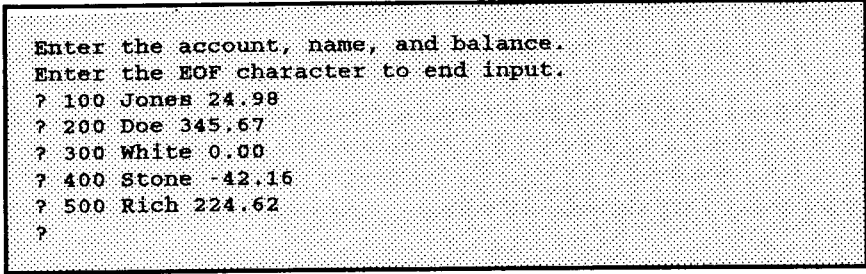


Fig. 11.3 Cómo crear un archivo secuencial.

de cuenta. Este programa supone que el usuario escribe los registros en orden por número de cuenta. En un sistema de cuentas por cobrar más completo, se incluiría una capacidad de ordenamiento, de tal forma que el usuario pudiera introducir los registros en cualquier orden. Los registros a continuación serían clasificados y escritos en el archivo.

Examinemos ahora este programa. El enunciado

FILE *cfPtr;

establece que **cfPtr** es un apuntador a una estructura **FILE**. El programa C administra cada archivo con una estructura **FILE** por separado. Para utilizar archivos el programador no necesita

saber los detalles específicos de la estructura **FILE**. Pronto veremos precisamente cómo la estructura **FILE** lleva indirectamente al bloque de control de archivos del sistema operativo (FCB) correspondiente a un archivo.

Sugerencia de portabilidad 11.1

La estructura FILE depende del sistema operativo (es decir, los miembros de la estructura varían de un sistema a otro, según la forma en que cada sistema maneja sus archivos).

Cada archivo abierto debe tener un apuntador declarado por separado del tipo **FILE**, que es utilizado para referirse al archivo. La línea

```
if ((cfPtr = fopen("clients.dat", "w")) == NULL)
```

nombra el archivo —**clients.dat**— para ser utilizado por el programa y establece una “línea de comunicación” con el archivo. El apuntador de archivo **cfPtr** es asignado a un apuntador a la estructura **FILE** para el archivo abierto con **fopen**. La función **fopen** toma dos argumentos: un nombre de archivo y un *modo de archivo abierto*. El modo de archivo abierto **“w”** indica que el archivo debe de ser abierto para *escritura*. Si el archivo no existe y es abierto para escritura, **fopen** crea el archivo. Si un archivo existente es abierto para escritura, el contenido del archivo es descartado sin advertencia. En el programa, la estructura **if** se utiliza para determinar si el apuntador al archivo **cfPtr** es **NULL** (es decir, el archivo no está abierto). Si es **NULL**, se imprime un mensaje de error y el programa termina. De lo contrario, la entrada es procesada y escrita al archivo.

Error común de programación 11.1

Abrir un archivo existente para escritura (“w”) cuando, de hecho, el usuario desea conservar el archivo; el contenido del archivo se descartará sin advertencia.

Error común de programación 11.2

Olvidar abrir un archivo antes de intentar hacer referencia a él en un programa.

El programa le solicita al usuario que introduzca los varios campos de cada registro, o que introduzca fin de archivo cuando esté completa la entrada de datos. La figura 11.4 enlista las combinaciones de teclas para introducir fin de archivo en varios sistemas de computación.

La línea

```
while (!feof(stdin))
```

utiliza la función **feof** para determinar si el *indicador de fin de archivo* está definido para el archivo al que se refiere **stdin**. El indicador de fin de archivo le informa al programa que ya no hay más datos a procesarse. En el programa de la figura 11.3, el indicador de fin de archivo está

Sistema de computación	Combinación de teclas
Sistemas UNIX	<return><ctrl>d
IBM PC y compatibles	<ctrl>z
Macintosh	<ctrl>d
VAX (VMS)	<ctrl>z

Fig. 11.4 Combinaciones de teclas de fin de archivo correspondientes a varios sistemas populares de computación.

definido para la entrada estándar cuando el usuario introduce la combinación de teclas de fin de archivo. El argumento de la función `feof` es un apuntador al archivo bajo prueba en relación con el indicador de fin de archivo (en este caso `stdin`). La función regresa un valor no cero (verdadera) una vez definido el indicador de fin de archivo; de lo contrario regresa cero. La estructura `while`, que en este programa incluye la llamada `feof`, se continuará ejecutando en tanto no se defina el indicador de fin de archivo.

El enunciado

```
fprintf(cfPtr, "%d %s %.2f\n", account, name, balance);
```

escribe datos al archivo `clients.dat`. Los datos pueden ser recuperados más tarde mediante un programa diseñado para leer el archivo (vea la sección 11.5). La función `fprintf` es equivalente a `printf`, excepto que `fprintf` también recibe como argumento un apuntador de archivo para el archivo al cual se escribirán los datos.

Error común de programación 11.3

Usar el apuntador de archivo incorrecto para referirse a un archivo.

Práctica sana de programación 11.1

Asegúrese que en un programa las llamadas a las funciones de procesamiento de archivos contienen los apuntadores de archivo correctos.

Después de que el usuario haya introducido el fin de archivo, el programa cierra el archivo `clients.dat` utilizando `fclose` y termina. La función `fclose` también recibe el apuntador de archivo (en vez del nombre del archivo) como un argumento. Si no se llama a la función `fclose` en forma explícita, normalmente cuando la ejecución del programa termine el sistema operativo cerrará el archivo. Esto es un ejemplo de “buena administración interna” del sistema operativo.

Práctica sana de programación 11.2

Cierre cada archivo en forma explícita, tan pronto sepa que el programa ya no hará otra vez referencia al archivo.

Sugerencia de rendimiento 11.1

Cerrar un archivo puede liberar recursos que están siendo esperados por otros usuarios o programas.

En la ejecución de muestra del programa de la figura 11.3, el usuario introduce información correspondiente a cinco cuentas, y a continuación escribe el fin de archivo, para indicar que la entrada de datos está completa. La ejecución de muestra no pone de manifiesto cómo aparecen realmente los registros de datos dentro del archivo. A fin de verificar que el archivo haya sido creado exitosamente, en la siguiente sección introducimos un programa que lee el archivo e imprime su contenido.

En la figura 11.5 se ilustra la relación entre los apuntadores `FILE`, las estructuras `FILE` y los FCB en memoria. Cuando se abre el archivo “`clients.dat`”, se copia un FCB para dicho archivo en la memoria. La figura muestra la conexión entre el apuntador de archivo regresado por `fopen` y el FCB utilizado por el sistema operativo para la administración del archivo.

Los programas no pueden procesar ningún archivo, un archivo o varios archivos. Cada archivo utilizado en un programa deberá tener un nombre único, y deberá tener un apuntador de archivo distinto regresado por `fopen`. Todas las funciones de procesamiento de archivos subsecuentes después de su apertura deberán referirse al archivo utilizando el apuntador de archivo apropiado.

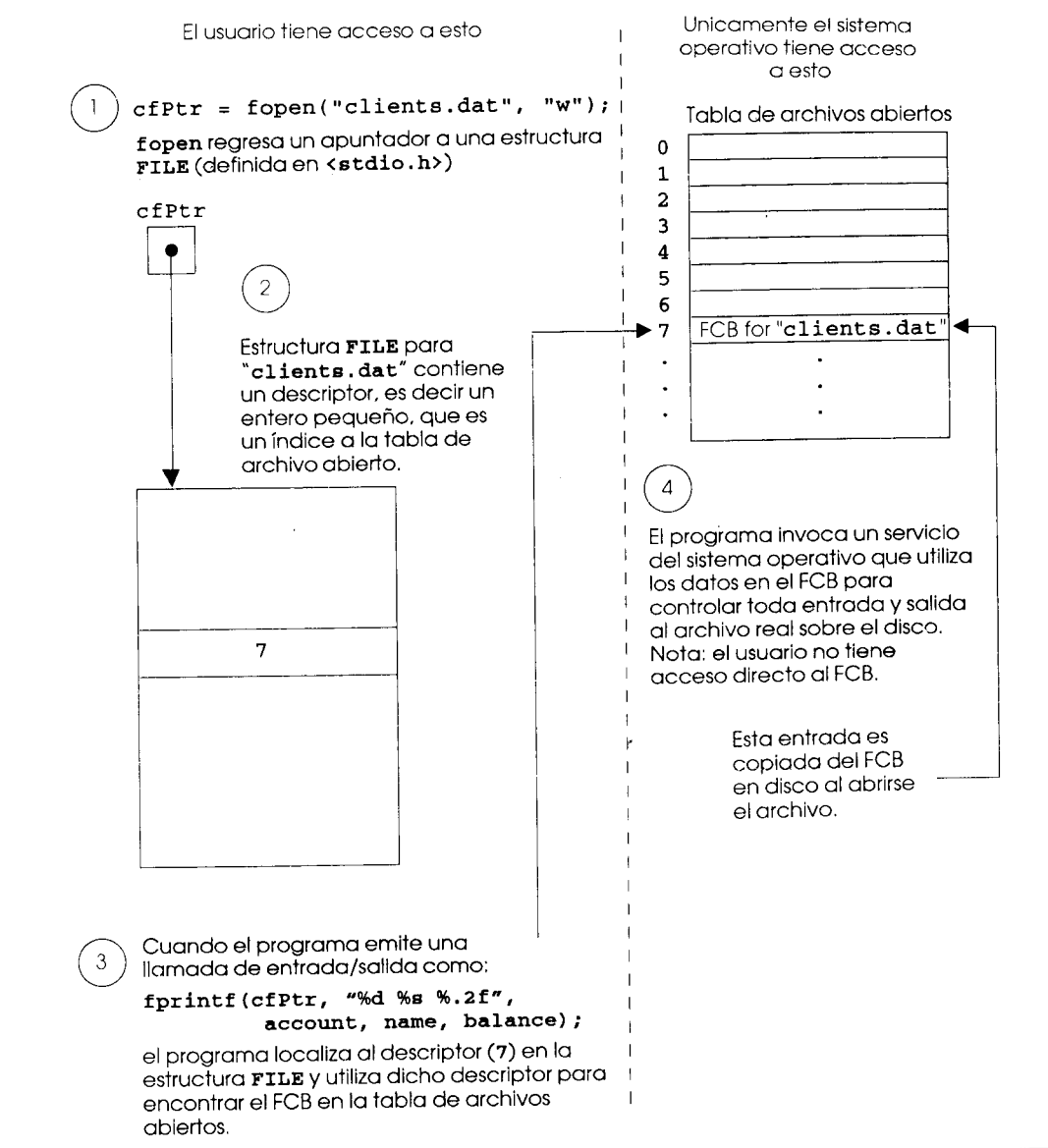


Fig. 11.5 Relación entre los apuntadores `FILE` las estructuras `FILE` y los FCB.

Los archivos pueden ser abiertos en uno de varios modos. Para crear un archivo, o para descartar el contenido de un archivo antes de escribir datos, abra el archivo para escritura (“`w`”). Para leer un archivo existente, ábralo para lectura (“`r`”). Para añadir registros al final de un archivo existente, abra el archivo para agregar (“`a`”). Para abrir a un archivo de tal forma que pueda ser escrito y leído, abra el archivo para actualizar en uno de los tres modos de actualización —“`r+`”, “`w+`” o “`a+`”. El modo “`r+`” abre un archivo para lectura y escritura. El modo “`w+`” genera un

archivo para lectura y escritura. Si el archivo ya existe, el archivo es abierto y el contenido actual de dicho archivo se descarta. El modo **"a+"** abre un archivo para lectura y escritura —toda escritura se efectuará al final del archivo. Si el archivo no existe será creado.

Si al abrir un archivo en cualquiera de los modos anteriores ocurre un error, **fopen** regresará **NULL**. Algunos errores posibles son:

Error común de programación 11.4

Abrir para lectura un archivo no existente.

Error común de programación 11.5

Abrir un archivo para lectura o escritura sin haber obtenido los derechos de acceso al archivo apropiados (esto depende del sistema operativo).

Error común de programación 11.6

Abrir un archivo para escritura cuando no hay espacio disponible en disco. En la figura 11.6 se enlistan los modos de apertura de archivos.

Error común de programación 11.7

*Puede llevar a errores devastadores abrir un archivo utilizando el modo de archivo incorrecto. Por ejemplo, abrir un archivo en el modo de escribir (**"w"**) cuando debería haberse abierto en modo de actualizar (**"r+"**) hace que sea descartado todo el contenido del archivo.*

Práctica sana de programación 11.3

Abra un archivo únicamente para lectura (y no para actualizar), si el contenido del archivo no debe modificarse. Esto evitará cambios no intencionales en el contenido del archivo. Este es otro ejemplo del principio del mínimo privilegio.

11.5 Cómo leer datos de un archivo de acceso secuencial

Los datos se almacenan en archivos, de tal forma que cuando sea necesario puedan ser recuperados para su proceso. La sección anterior demostró cómo crear un archivo para acceso secuencial. En esta sección, analizamos como leer secuencialmente los datos de un archivo.

Modo	Descripción
r	Abrir una archivo para lectura.
w	Crear un archivo para escritura. Si el archivo ya existe, se descarta el contenido actual.
a	Agregar; abrir o crear un archivo para escribir al final del mismo.
r+	Abrir un archivo para actualizar (leer y escribir).
w+	Crear un archivo para actualizar. Si el archivo ya existe, se descarta el contenido actual.
a+	Agregar; abrir o crear un archivo para actualizar; la escritura se efectuará al final del archivo.

Fig. 11.6 Modos de apertura de archivo.

El programa de la figura 11.7 lee registros del archivo **"clients.dat"** creados por el programa de la figura 11.3 e imprime el contenido de los registros. El enunciado

FILE *cfPtr;

indica que **cfPtr** es un apuntador a un **FILE**. La línea

if ((cfPtr = fopen("clients.dat", "r")) ==NULL)

intenta abrir el archivo **"clients.dat"** para lectura (**'r'**) y determina si el archivo se ha abierto con éxito (es decir, que **fopen** no regresa **NULL**). El enunciado

fscanf(cfPtr, "%d%s%f, &account, name, &balance);

lee un "registro" del archivo. La función **fscanf** es equivalente a la función **scanf**, salvo que **fscanf** recibe como argumento un apuntador a un archivo para el archivo del cual se van a leer datos. Después de que el enunciado anterior es ejecutado por primera vez, **account** tendrá el valor 100, **name** tendrá el valor **"Jones"**, y **balance** tendrá el valor **24.98**. Cada vez que se ejecute el segundo enunciado **fscanf**, se leerá otro registro del archivo y **account**, **name** y **balance** tomarán nuevos valores. Cuando se llegue al final del archivo, éste se cerrará y el programa terminará.

```
/* Reading and printing a sequential file */
#include <stdio.h>

main()
{
    int account;
    char name[30];
    float balance;
    FILE *cfPtr;    /* cfPtr = clients.dat file pointer */

    if ((cfPtr = fopen("clients.dat", "r")) == NULL)
        printf("File could not be opened\n");
    else {
        printf("%-10s%-13s\n", "Account", "Name", "Balance");
        fscanf(cfPtr, "%d%s%f", &account, name, &balance);

        while (!feof(cfPtr)) {
            printf("%-10d%-13s%7.2f\n", account, name, balance);
            fscanf(cfPtr, "%d%s%f", &account, name, &balance);
        }

        fclose(cfPtr);
    }

    return 0;
}
```

Fig. 11.7 Cómo leer e imprimir un archivo secuencial (parte 1 de 2).

Account	Name	Balance
100	Jones	24.98
200	Doe	345.67
300	White	0.00
400	Stone	-42.16
500	Rich	224.62

Fig. 11.7 Cómo leer e imprimir un archivo secuencial (parte 2 de 2).

Para recuperar secuencialmente datos de un archivo, un programa normalmente empieza a leer a partir del principio del archivo, y lee todos los datos en forma consecutiva, hasta que encuentra los datos deseados. Durante la ejecución de un programa pudiera ser deseable procesar los datos secuencialmente en un archivo varias veces (a partir del principio del archivo). Un enunciado como

```
rewind(cfPtr);
```

genera un *apuntador de posición de archivo* del programa —que indica el número del siguiente byte del archivo a leerse o a escribirse— a que se recoloca al principio del archivo (es decir en el byte 0) al cual apunta `cfPtr`. El apuntador de posición de archivo no es realmente un apuntador. Más bien es un valor entero que especifica la posición de byte en el archivo, en el cual ocurrirá la siguiente lectura o escritura. Esto a veces se denomina el *desplazamiento de archivo*. El apuntador de posición de archivo es un miembro de la estructura `FILE` asociado con cada archivo.

Ahora introducimos un programa (figura 11.8) que le permite a un gerente de crédito obtener listas de clientes con saldo 0 (es decir, clientes que no deben ningún dinero), clientes con saldos acreedores (es decir, clientes a los cuales la empresa les debe dinero), clientes con saldos deudores (es decir, clientes que le deben dinero a la empresa por bienes y servicios recibidos). Un saldo acreedor es una cantidad negativa; un saldo deudor es una cantidad positiva.

El programa despliega un menú y le permite al gerente de crédito introducir una de tres opciones para obtener información de crédito. La opción 1 produce una lista de cuentas con saldos en cero. La opción 2 produce una lista de cuentas con saldos acreedores. La opción 3 produce una lista de cuentas con saldos deudores. La opción 4 termina la ejecución del programa. Una salida de muestra se despliega en la figura 11.9.

Note que los datos en este tipo de archivo secuencial no pueden ser modificados sin riesgo de destruir otros datos dentro del archivo. Por ejemplo, si el nombre "White" necesitara ser modificado a "Worthington", el nombre antiguo simplemente no puede ser sobrescrito. El registro para White fue escrito al archivo como

```
300 White 0.00
```

Si el registro se reescribe utilizando el nuevo nombre empezando en la misma posición en el archivo, el registro sería

```
300 Worthington 0.00
```

El nuevo registro es más largo que el original. Los caracteres más allá de la segunda "o" de "Worthington" sobrescribirían el principio del siguiente registro secuencial en el archivo. El

```
/* Credit inquiry program */
#include <stdio.h>

main()
{
    int request, account;
    float balance;
    char name[30];
    FILE *cfPtr;

    if ((cfPtr = fopen("clients.dat", "r")) == NULL)
        printf("File could not be opened\n");
    else {
        printf("Enter request\n");
        printf(" 1 - List accounts with zero balances\n");
        printf(" 2 - List accounts with credit balances\n");
        printf(" 3 - List accounts with debit balances\n");
        printf(" 4 - End of run\n? ");
        scanf("%d", &request);

        while (request != 4) {
            fscanf(cfPtr, "%d%s%f", &account, name, &balance);

            switch (request) {
                case 1:
                    printf("\nAccounts with zero balances:\n");

                    while (!feof(cfPtr)) {

                        if (balance == 0)
                            printf("%-10d%-13s%7.2f\n",
                                account, name, balance);

                        fscanf(cfPtr, "%d%s%f",
                            &account, name, &balance);
                    }

                    break;
                case 2:
                    printf("\nAccounts with credit balances:\n");

                    while (!feof(cfPtr)) {

                        if (balance < 0)
                            printf("%-10d%-13s%7.2f\n",
                                account, name, balance);

                        fscanf(cfPtr, "%d%s%f",
                            &account, name, &balance);
                    }

                    break;
            }
        }
    }
}
```

Fig. 11.8 Programa de consulta de crédito (parte 1 de 2).

```
case 3:
    printf("\nAccounts with debit balances:\n");
    while (!feof(cfPtr)) {
        if (balance > 0)
            printf("%-10d%-13s%7.2f\n",
                account, name, balance);
        fscanf(cfPtr, "%d%s%f",
            &account, name, &balance);
    }
    break;

rewind(cfPtr);
printf("\n? ");
scanf("%d", &request);
}

printf("End of run.\n");
fclose(cfPtr);
}

return 0;
}
```

Fig. 11.8 Programa de consulta de crédito (parte 2 de 2).

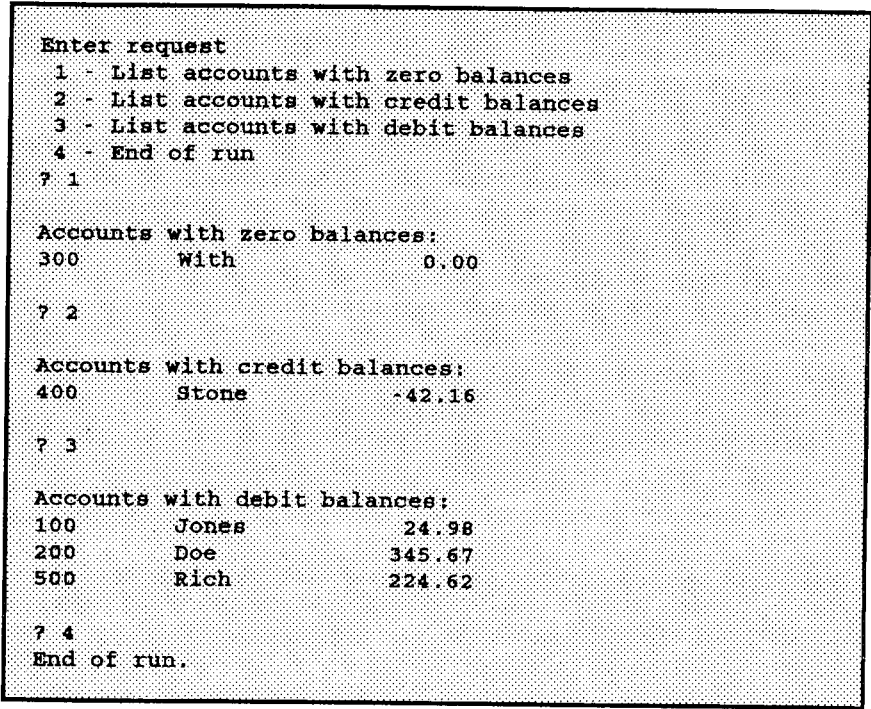


Fig. 11.9 Salida de muestra del programa de consulta de crédito de la figura 11.8.

problema aquí es que en el modelo de entrada/salida con formato utilizando `fprintf` y `fscanf`, los campos —y por lo tanto los registros— pueden variar de tamaño. Por ejemplo, 7, 14, -117, 2074, y 27383 son todos `int` almacenados internamente en el mismo número de bytes, pero imprimen en la pantalla o `fprintf` en el disco como campos de tamaño diferente.

Por lo tanto, el acceso secuencial mediante `fprint` y `fscanf` normalmente no se utiliza para actualizar registros en su sitio. En vez de ello, usualmente la totalidad del archivo se vuelve a escribir. Para llevar a cabo la modificación de nombre anteriormente citada, los registros anteriores a 300 `White 0.00` en un archivo como éste, de acceso secuencial, serían copiados a un nuevo archivo, sería escrito el nuevo registro, y los registros existentes después de 300 `White 0.00` serían vueltos a copiar al nuevo archivo. Esto significa el procesamiento de todos los registros en el archivo para simplemente actualizar un registro.

11.6 Archivos de acceso directo

Como hemos indicado anteriormente, los registros en un archivo creados con la función de salida `fprintf` con formato, no necesariamente son de la misma longitud. Sin embargo, los registros individuales de un *archivo de acceso directo* normalmente son de longitud fija y se puede tener acceso a ellos directamente (y por lo tanto rápidamente) sin tener que buscar a través de otros registros. Esto hace que los archivos de acceso directo sean apropiados para sistemas de reservación de aerolíneas, sistemas bancarios, sistemas de punto de venta y otros tipos de *sistemas de procesamiento de transacciones*, que requieren de acceso rápido a datos específicos. Existen otras formas para poner en funcionamiento archivos de acceso directo, pero limitaremos nuestro análisis a este enfoque sencillo de utilización de registros de longitud fija.

Dado que en un archivo de acceso directo todos los registros normalmente tienen la misma longitud, la posición exacta de un registro en relación con el principio del archivo puede ser calculada como una función del registro clave. Pronto veremos como esto facilita el acceso inmediato a registros específicos, inclusive en archivos grandes.

En la figura 11.10 se ilustra una forma para poner en marcha un archivo de acceso directo. Un archivo como éste es similar a un tren de carga con muchos vagones —algunos vacíos y algunos con carga. Cada vagón en el tren tiene la misma longitud.

Los datos pueden ser insertos en un archivo de acceso directo sin destruir otros datos en el archivo. Los datos almacenados anteriormente también pueden ser actualizados o borrados, sin tener que reescribir todo el archivo. En las secciones que siguen explicaremos como se crea un archivo de acceso directo, como se introduce información, como se leen los datos tanto secuencial como directamente, como se actualizan los datos y como se borran aquellos datos que ya no se requieren.

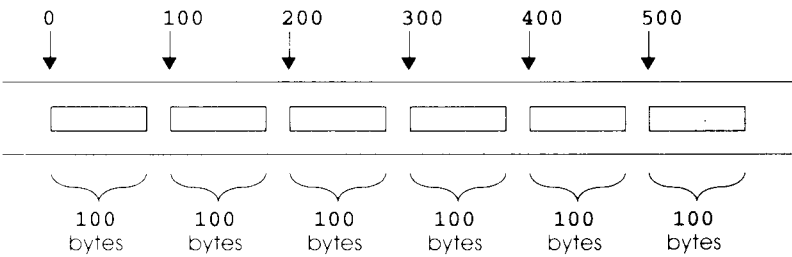


Fig. 11.10 Vista de un archivo de acceso directo con registros de longitud fija.

11.7 Cómo crear un archivo de acceso directo

La función **fwrite** transfiere a un archivo un número especificado de bytes empezando en una posición especificada de memoria. Los datos se escriben al principio de la posición en el archivo indicada mediante el apuntador de posición de archivo. La función **fread** transfiere un número especificado de bytes de la posición en el archivo, especificado por el apuntador de posición de archivo, a un área en memoria empezando a partir de una dirección especificada. Ahora, al escribir un entero, en vez de utilizar

```
fprintf(fPtr, "%d", number);
```

podría imprimir desde 1 dígito hasta un máximo de 11 dígitos (10 dígitos más un signo, cada uno de los cuales requiere 1 byte de almacenamiento) para un entero de 4 bytes, podemos utilizar

```
fwrite(&number, sizeof(int), 1, fPtr);
```

que siempre escribirá 4 bytes (o 2 bytes en un sistema con enteros de 2 bytes) de la variable **number** al archivo representado por **fPtr** (explicaremos en breve el argumento 1). Más adelante, **fread** podrá ser utilizado para leer 4 de estos bytes a la variable entera **number**. Aunque **fread** y **fwrite** leen y escriben datos, como son enteros, en tamaño fijo en vez de en formato de tamaño variable, los datos que manejan se procesan en formato “en bruto” de computadora (es decir, bytes de datos), en vez de en el formato legible para los seres humanos **printf** y **scanf**.

Las funciones **fwrite** y **fread** son capaces de leer y de escribir arreglos de datos hacia y desde el disco. El tercer argumento, tanto de **fread** como **fwrite** es el número de elementos en el arreglo que deberá ser leído del disco, o escrito al disco. La llamada anterior de función **fwrite** escribe un solo entero al disco, por lo que el tercer argumento es 1 (como si un elemento de un arreglo fuera escrito).

Los programas de procesamiento de archivos rara vez escriben un solo campo a un archivo. Normalmente, escriben un **struct** a la vez, como veremos en los ejemplos siguientes.

Considere el siguiente enunciado de problema:

Crear un sistema de procesamiento de crédito capaz de almacenar hasta 100 registros de longitud fija. Cada registro deberá estar formado de un número de cuenta, que será utilizado como registro clave, un apellido, un nombre y un saldo. El programa resultante deberá ser capaz de actualizar una cuenta, insertar un nuevo registro de cuenta, borrar una cuenta y enlistar todos los registros de cuenta en un archivo de texto con formato para su impresión. Utilice un archivo de acceso directo.

En las siguientes varias secciones se introducen las técnicas necesarias para crear el programa de procesamiento de crédito. El programa de la figura 11.11 muestra como abrir un archivo de acceso directo, definir un formato de registro utilizando un **struct**, escribir datos al disco, y cerrar el archivo. Este programa inicializa los 100 registros del archivo “**credit.dat**”, con **structs** vacíos utilizando la función **fwrite**. Cada **struct** vacío contiene 0 para el número de cuenta, **NULL** (representado por comillas vacías) para el apellido, **NULL** para el nombre y 0.0 para el saldo. El archivo se inicializa de esta forma para crear espacio en el disco en el cual se almacenará el archivo, y para que sea posible determinar si un registro contiene datos.

La función **fwrite** escribe un bloque (un número específico de bytes) de datos a un archivo. En nuestro programa, el enunciado

```
fwrite(&blankClient, sizeof(struct clientData), 1, cfPtr);
```

```
/* Creating a randomly accessed file sequentially */
#include <stdio.h>

struct clientData {
    int acctNum;
    char lastName[15];
    char firstName[10];
    float balance;
};

main()
{
    int i;
    struct clientData blankClient = {0, "", "", 0.0};
    FILE *cfPtr;

    if ((cfPtr = fopen("credit.dat", "w")) == NULL)
        printf("File could not be opened.\n");
    else {

        for (i = 1; i <= 100; i++)
            fwrite(&blankClient,
                sizeof(struct clientData), 1, cfPtr);

        fclose (cfPtr);
    }

    return 0;
}
```

Fig. 11.11 Cómo crear un archivo de acceso directo en forma secuencial.

hace que la estructura **blankClient** de tamaño **sizeof(struct clientData)** se escriba al archivo al cual apunta **cfPtr**. El operador **sizeof** regresa el tamaño en bytes del objeto contenido en los paréntesis (en este caso **struct clientData**). El operador **sizeof** es un operador unario en tiempo de compilación que regresa un entero no signado. El operador **sizeof** puede ser utilizado para determinar el tamaño en bytes de cualquier tipo o expresión de datos. Por ejemplo, **sizeof(int)** se utiliza para determinar si en una computadora en particular un entero está almacenado en 2 o en 4 bytes.

Sugerencia de rendimiento 11.2

*Muchos programadores piensan erróneamente que **sizeof** es una función, y que utilizándolo se genera una sobrecarga en tiempo de ejecución de una llamada de función. No existe tal sobrecarga, porque **sizeof** es un operador en tiempo de compilación.*

La función **fwrite** puede de hecho ser utilizada para escribir varios elementos de un arreglo de objetos. Para escribir varios elementos de arreglo, el programador proporciona un apuntador a un arreglo como primer argumento en la llamada **fwrite**, y especifica el número de elementos a escribirse como el tercer argumento en la llamada **fwrite**. En el enunciado anterior, **fwrite** fue utilizado para escribir un solo objeto que no era un elemento de arreglo. Escribir un solo objeto es el equivalente a escribir un elemento de un arreglo, de ahí el 1 en la llamada **fwrite**.

11.8 Cómo escribir datos directamente a un archivo de acceso directo

El programa de la figura 11.2 escribe datos al archivo "credit.dat". Utiliza la combinación de **fseek** y **fwrite** para almacenar datos en posiciones específicas dentro del archivo. La función **fseek** define el apuntador de posición de archivo a una posición específica dentro del archivo, y a continuación **fwrite** escribe los datos. Una ejecución de muestra aparece en la figura 11.13.

El enunciado

```
fseek(cfPtr, (accountNum - 1) * sizeof(struct clientData);
      SEEK_SET);
```

```
/* Writing to a random access file */
#include <stdio.h>

struct clientData {
    int acctNum;
    char lastName[15];
    char firstName[10];
    float balance;
};

main()
{
    FILE *cfPtr;
    struct clientData client;

    if ((cfPtr = fopen("credit.dat", "r+")) == NULL)
        printf("File could not be opened.\n");
    else {
        printf("Enter account number\n"
               " (1 to 100, 0 to end input)\n? ");
        scanf("%d", &client.acctNum);

        while (client.acctNum != 0) {
            printf("Enter lastname, firstname, balance\n? ");
            scanf("%s%s%f", &client.lastName,
                  &client.firstName, &client.balance);
            fseek(cfPtr, (client.acctNum - 1) *
                  sizeof(struct clientData), SEEK_SET);
            fwrite(&client, sizeof(struct clientData), 1, cfPtr);
            printf("Enter account number\n? ");
            scanf("%d", &client.acctNum);
        }

        fclose(cfPtr);

        return 0;
    }
}
```

Fig. 11.12 Cómo escribir datos directamente a un archivo de acceso directo.

```
Enter account number (1 to 100, 0 to end input)
? 37
Enter lastname, firstname, balance
? Barker Doug 0.00
Enter account number
? 29
Enter lastname, firstname, balance
? Brown Nancy -24.54
Enter account number
? 96
Enter lastname, firstname, balance
? Stone Sam 34.98
Enter account number
? 88
Enter lastname, firstname, balance
? Smith Dave 258.34
Enter account number
? 33
Enter lastname, firstname, balance
? Dunn Stacey 314.33
Enter account number
? 0
```

Fig. 11.13 Ejecución de muestra del programa de la figura 11.12.

posiciona el apuntador de posición de archivo para el archivo referenciado por **cfPtr**, a la posición de bytes calculada por **(accountNum - 1) * sizeof(struct clientData)**; el valor de esta expresión se conoce como el *desplazamiento*. Dado que el número de cuenta está entre 1 y 100, pero las posiciones de bytes en el archivo empiezan con 0, al calcular la posición de bytes dentro del registro se resta 1 del número de cuenta. Entonces, para el registro 1, el apuntador de posición de archivo se define al byte 0 del archivo. La constante simbólica **SEEK_SET** indica que el apuntador de posición de archivo está colocado en relación con el principio del archivo, en la cantidad del desplazamiento. Como indica el enunciado anterior, una búsqueda para el número de cuenta 1 en el archivo define el apuntador de posición de archivo al principio del archivo, porque la posición de bytes calculada es 0. La figura 11.14 ilustra el apuntador de archivo que hace referencia a una estructura **FILE** en memoria. El apuntador de posición de archivo indica que el siguiente byte a leerse o escribirse está a 5 bytes del principio del archivo.

El estándar ANSI muestra la función prototipo para **fseek** como

```
int fseek(FILE *stream, long int offset, int whence);
```

donde **offset** es el número de bytes a partir de la posición **whence** en el archivo al cual apunta **stream**. El argumento **whence** puede tener uno de tres valores —**SEEK_SET**, **SEEK_CUR** o **SEEK_END**— indicando la posición en el archivo a partir del cual se inicia la búsqueda. **SEEK_SET** indica que la búsqueda se inicia al principio del archivo; **SEEK_CUR** indica que la búsqueda se inicia en la posición actual en el archivo; y **SEEK_END** indica que la búsqueda se inicia en el final del archivo. Estas tres constantes simbólicas se definen en el archivo de cabecera **stdio.h**.

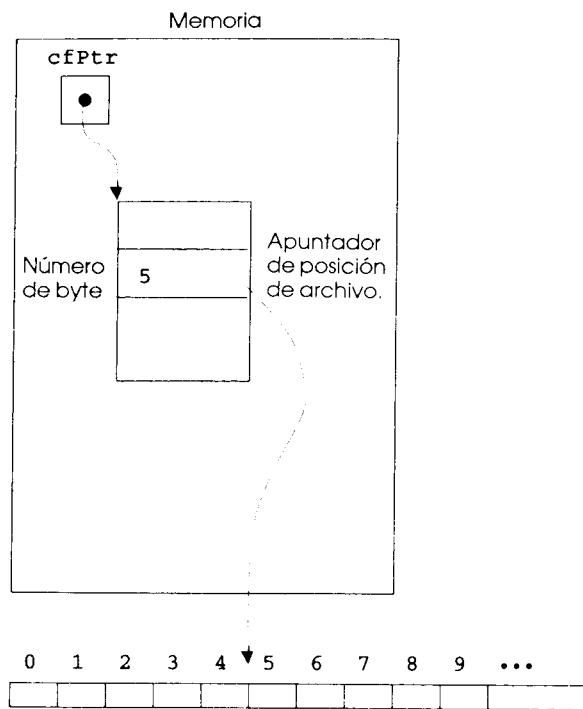


Fig. 11.14 Apuntador de posición de archivo, indicando un desplazamiento de 5 bytes a partir del principio del archivo.

11.9 Cómo leer datos directamente de un archivo de acceso directo

La función `fread` lee un número especificado de bytes de un archivo a la memoria. Por ejemplo, el enunciado

```
fread(&client, sizeof(struct clientData), 1, cfPtr);
```

lee el número de bytes determinado por `sizeof(struct clientData)` correspondiente al archivo referenciado por `cfPtr` y almacena el dato en la estructura `client`. Los bytes son leídos de la posición en el archivo especificado por el apuntador de posición de archivo. La función `fread` puede ser utilizada para leer varios elementos de arreglo de tamaño fijo, proporcionando un apuntador al arreglo en el cual los elementos se almacenarán, e indicando el número de elementos a leerse. El enunciado anterior especifica que un elemento deberá ser leído. Para poder leer más de un elemento, especifique el número de elementos en el tercer argumento del enunciado `fread`.

El programa de la figura 11.15 lee en forma secuencial todos los registros en el archivo `credit.dat`, determina si cada uno de dichos registros contiene datos, e imprime los datos con formato correspondientes a los registros que contengan datos. La función `feof` determina cuando se alcanza el fin de archivo, y la función `fread` transfiere los datos del disco a la estructura `client` de `clientData`.

```
/* Reading a random access file sequentially */
#include <stdio.h>

struct clientData {
    int acctNum;
    char lastName[15];
    char firstName[10];
    float balance;
};

main()
{
    FILE *cfPtr;
    struct clientData client;

    if ((cfPtr = fopen("credit.dat", "r")) == NULL)
        printf("File could not be opened.\n");
    else {
        printf("%-6s%-16s%-11s%10s\n", "Acct", "Last Name",
            "First Name", "Balance");

        while (!feof(cfPtr)) {
            fread(&client, sizeof(struct clientData), 1, cfPtr);

            if (client.acctNum != 0)
                printf("%-6d%-16s%-11s%10.2f\n",
                    client.acctNum, client.lastName,
                    client.firstName, client.balance);
        }

        fclose(cfPtr);

        return 0;
    }
}
```

Acct	Last Name	First Name	Balance
29	Brown	Nancy	-24.54
33	Dunn	Stacey	314.33
37	Barker	Doug	0.00
88	Smith	Dave	258.34
96	Stone	Sam	34.98

Fig. 11.5 Cómo leer secuencialmente un archivo de acceso directo.

11.10 Estudio de caso: un programa de procesamiento de transacciones

Ahora presentamos un programa sustancial de procesamiento de transacciones, que utiliza archivos de acceso directo. El programa mantiene información de cuentas de un banco. El programa actualiza las cuentas existentes, añade cuentas nuevas, borra cuentas y almacena un enlistado de todas las cuentas actuales, en un archivo de texto para su impresión. Suponemos que el programa de la figura 11.11 ha sido ejecutado para crear el archivo `credit.dat`.

El programa tiene cinco opciones. La opción 1 llama a la función `textFile` para almacenar una lista con formato de todas las cuentas en un archivo de texto llamado `accounts.txt`, que pudiera ser impreso más adelante. La función utiliza `fread` y las técnicas de acceso secuencial a archivo utilizadas en el programa de la figura 11.15. Después de seleccionar la opción 1 el archivo `accounts.txt` contiene:

Acct	Last Name	First Nam	Balance
29	Brown	Nancy	-24.54
33	Dunn	Stacey	314.33
37	Barker	Doug	0.00
88	Smith	Dave	258.34
96	Stone	Sam	34.98

La opción 2 llama a la función `updateRecord` para actualizar una cuenta. La función únicamente actualizará un registro ya existente, por lo que la función primero averiguará si el registro especificado por el usuario está vacío. El registro se lee a la estructura `client` utilizando a `fread`, y a continuación se utiliza `strcmp` para determinar si es `NULL` el miembro `lastName` de la estructura `client`. De ser así, el registro no contiene información, y se imprime un mensaje indicando que el registro está vacío. A continuación se despliegan las selecciones de menú. Si el registro contiene información, la función `updateRecord` introduce la cantidad de transacción, calcula el nuevo saldo, y vuelve a escribir el registro al archivo. Una salida típica correspondiente a la opción 2 es:

```
Enter account to update (1 - 100): 37
37  Barker      Doug      0.00

Enter charge (+) or payment (-): +87.99
37  Barker      Doug      87.99
```

La opción 3 llama a la función `newRecord` a fin de añadir una nueva cuenta al archivo. Si el usuario introduce un número de cuenta correspondiente a una cuenta existente, `newRecord` despliega un mensaje de error, indicando que el registro ya contiene información, y las selecciones de menú se vuelven a imprimir. Esta función utiliza para añadir una nueva cuenta el mismo proceso que en el programa de la figura 11.12. Una salida típica para la opción 3 es

```
Enter new account number (1 - 100): 22
Enter lastname, firstname, balance
? Johnston Sarah 247.45
```

La opción 4 llama a la función `deleteRecord` para borrar un registro del archivo. El borrado se lleva a cabo solicitándole al usuario el número de cuenta y volviendo a inicializar el registro. Si la cuenta no contiene información, `deleteRecord` despliega un mensaje de error, indicando que la cuenta no existe. La opción 5 termina la ejecución del programa. El programa se muestra en la figura 11.16. Advierta que el archivo `"credit.dat"` se abre para actualizar (lectura y escritura) mediante el modo `"r+"`.

```
/* This program reads a random access file sequentially, *
 * updates data already written to the file, creates new *
 * data to be placed in the file, and deletes data *
 * already in the file. */

#include <stdio.h>

struct clientData {
    int acctNum;
    char lastName[15];
    char firstName[10];
    float balance;
};

int enterChoice(void);
void textFile(FILE *);
void updateRecord(FILE *);
void newRecord(FILE *);
void deleteRecord(FILE *);

main()
{
    FILE *cfPtr;
    int choice;

    if ((cfPtr = fopen("credit.dat", "r+")) == NULL)
        printf("File could not be opened.\n");
    else {

        while ((choice = enterChoice()) != 5) {

            switch (choice) {
                case 1:
                    textFile(cfPtr);
                    break;
                case 2:
                    updateRecord(cfPtr);
                    break;
                case 3:
                    newRecord(cfPtr);
                    break;
                case 4:
                    deleteRecord(cfPtr);
                    break;
            }

        }

        fclose(cfPtr);
        return 0;
    }
}
```

Fig. 11.16 Programa de cuentas de banco (parte 1 de 4).

```

void textFile(FILE *readPtr)
{
    FILE *writePtr;
    struct clientData client;

    if ((writePtr = fopen("accounts.txt", "w")) == NULL)
        printf("File could not be opened.\n");
    else {
        rewind(readPtr);
        fprintf(writePtr, "%-6s%-16s%-11s%10s\n",
            "Acct", "Last Name", "First Name", "Balance");

        while (!feof(readPtr)) {
            fread(&client, sizeof(struct clientData), 1, readPtr);

            if (client.acctNum != 0)
                fprintf(writePtr, "%-6d%-16s%-11s%10.2f\n",
                    client.acctNum, client.lastName,
                    client.firstName, client.balance);
        }

        fclose(writePtr);
    }

    void updateRecord(FILE *fPtr)
    {
        int account;
        float transaction;
        struct clientData client;

        printf("Enter account to update (1 - 100): ");
        scanf("%d", &account);
        fseek(fPtr, (account - 1) * sizeof(struct clientData),
            SEEK_SET);
        fread(&client, sizeof(struct clientData), 1, fPtr);

        if (client.acctNum == 0)
            printf("Account #d has no information.\n", account);
        else {
            printf("%-6d%-16s%-11s%10.2f\n\n",
                client.acctNum, client.lastName,
                client.firstName, client.balance);
            printf("Enter charge (+) or payment (-): ");
            scanf("%f", &transaction);
            client.balance += transaction;
            printf("%-6d%-16s%-11s%10.2f\n",
                client.acctNum, client.lastName,
                client.firstName, client.balance);
            fseek(fPtr, (account - 1) * sizeof(struct clientData),
                SEEK_SET);
            fwrite(&client, sizeof(struct clientData), 1, fPtr);
        }
    }
}

```

Fig. 11.16 Programa de cuentas de banco (parte 2 de 4).

```

void deleteRecord(FILE *fPtr)
{
    struct clientData client, blankClient = {0, "", "", 0};
    int accountNum;

    printf("Enter account number to delete (1 - 100): ");
    scanf("%d", &accountNum);
    fseek(fPtr, (accountNum - 1) * sizeof(struct clientData),
        SEEK_SET);
    fread(&client, sizeof(struct clientData), 1, fPtr);

    if (client.acctNum == 0)
        printf("Account %d does not exist.\n", accountNum);
    else {
        fseek(fPtr, (accountNum - 1) * sizeof(struct clientData),
            SEEK_SET);
        fwrite(&blankClient, sizeof(struct clientData), 1, fPtr);
    }
}

void newRecord(FILE *fPtr)
{
    struct clientData client;
    int accountNum;
    printf("Enter new account number (1 - 100): ");
    scanf("%d", &accountNum);
    fseek(fPtr, (accountNum - 1) * sizeof(struct clientData),
        SEEK_SET);
    fread(&client, sizeof(struct clientData), 1, fPtr);

    if (client.acctNum != 0)
        printf("Account #d already contains information.\n",
            client.acctNum);
    else {
        printf("Enter lastname, firstname, balance\n? ");
        scanf("%s%s%f", &client.lastName, &client.firstName,
            &client.balance);
        client.acctNum = accountNum;
        fseek(fPtr, (client.acctNum - 1) *
            sizeof(struct clientData), SEEK_SET);
        fwrite(&client, sizeof(struct clientData), 1, fPtr);
    }
}

```

Fig. 11.16 Programa de cuentas de banco (parte 3 de 4).

Resumen

- Todos los elementos de datos procesados por una computadora se reducen a combinaciones de ceros y de unos.
- En una computadora el elemento más pequeño de datos puede asumir el valor 0 o el valor 1. Este elemento de datos se conoce como un bit (abreviatura de "binary digit" —un dígito que puede asumir uno de dos valores).

```
int enterChoice(void)
{
    int menuChoice;

    printf("\nEnter your choice\n"
        "1 - store a formatted text file of accounts called\n"
        "    \"accounts.txt\" for printing\n"
        "2 - update an account\n"
        "3 - add a new account\n"
        "4 - delete an account\n"
        "5 - end program\n? ");
    scanf("%d", &menuChoice);
    return menuChoice;
}
```

Fig. 11.16 Programa de cuentas de banco (parte 4 de 4).

- Los dígitos, las letras y los símbolos especiales se conocen como caracteres. El conjunto de todos los caracteres que pueden ser utilizados para escribir programas y representar elementos de datos en una computadora en particular se conoce como el conjunto de caracteres de la computadora. Cada uno de los caracteres en el conjunto de caracteres de la computadora se representa como un patrón de ocho 1s y 0s (conocido como un byte).
- Un campo es un grupo de caracteres que contiene significado.
- Un registro es un grupo de campos relacionados.
- Por lo menos un campo es seleccionado normalmente en cada registro como registro clave. El registro clave identifica un registro como perteneciente a una persona o entidad particular.
- El tipo de organización más popular para registros en un archivo, se conoce como archivo de acceso secuencial, en el cual se tiene acceso a registros en forma consecutiva hasta que son localizados los datos deseados.
- Un grupo de archivos relacionados a veces se llama una base de datos. Un conjunto o colección de programas diseñados para crear y administrar bases de datos se conoce como un sistema de administración de bases de datos (DBMS, por database management system).
- C considera cada archivo como simplemente un flujo secuencial de bytes.
- Al empezar la ejecución de un programa C abre automáticamente tres archivos y sus flujos asociados —entrada estándar, salida estándar y error estándar.
- Los apuntadores de archivo asignados a la entrada estándar, a la salida estándar y al error estándar son `stdin`, `stdout` y `stderr`, respectivamente.
- La función `fgetc` lee un carácter a partir de un archivo especificado.
- La función `fputc` escribe un carácter a un archivo especificado.
- La función `fgets` lee una línea de un archivo especificado.
- La función `fputs` escribe una línea en un archivo especificado.
- **FILE** es un tipo de estructura definida en el archivo de cabecera `stdio.h`. Para poder utilizar archivo el programador no necesita saber los detalles específicos de esta estructura. Conforme se abre un archivo, se regresa un apuntador a la estructura **FILE** del archivo.

- La función **fopen** toma dos argumentos— un nombre de archivo y un modo del archivo abierto— y abre el archivo. Si el archivo existe, el contenido del archivo se descarta sin advertencia. Si el archivo no existe y el archivo está siendo abierto para escribir, **fopen** crea el archivo.
- La función **feof** determina si ha sido definido el indicador de fin de archivo para el mismo.
- La función **fprintf** es equivalente a **printf**, excepto que **printf** recibe como argumento un apuntador al archivo hacia el cual se escribirán los datos.
- La función **fclose** cierra el archivo al cual apunta su argumento.
- Para crear un archivo, o para descartar el contenido de un archivo antes de escribir datos, abra el archivo para escritura ("**w**"). Para leer un archivo existente, ábralo para lectura ("**r**"). Para añadir registros al final de un archivo existente, abra el archivo para agregar ("**a**"). Para abrir un archivo de tal forma que pueda ser escrito y leído, abra el archivo para actualizar en alguno de los tres modos de actualización —"**r+**", "**w+**" o "**a+**". El modo "**r+**" simplemente abre el archivo para lectura y escritura. El modo "**w+**" crea el archivo si no existe, y si existe descarta el contenido actual del archivo. En el modo "**a+**" se crea el archivo si no existe, y la escritura se efectúa al final del archivo.
- La función **fscanf** es equivalente a **scanf**, excepto que **fscanf** recibe como argumento un apuntador al archivo (normalmente distinto a `stdin`) a partir del cual se leerán los datos.
- La función **rewind** hace que el programa recolocque el apuntador de posición del archivo correspondiente al archivo especificado al principio del mismo.
- Se utiliza el procesamiento de acceso directo de archivos para tener acceso directo a registros.
- Para facilitar el acceso directo, los datos se almacenan en registros de longitud fija. Dado que todos los registros son de la misma longitud, la computadora puede rápidamente calcular (como una función del registro clave) la posición exacta de un registro en relación con el principio del archivo.
- Los datos pueden ser añadidos fácilmente a un archivo de acceso directo sin destruir otros datos en el archivo. Los datos previamente almacenados en un archivo con registros de longitud fija, también pueden ser modificados y borrados sin tener que reescribir todo el archivo.
- La función **fwrite** escribe un bloque (número específico de bytes) de datos a un archivo.
- El operador en tiempo de compilación **sizeof** regresa el tamaño en bytes de su operando.
- La función **fseek** establece el apuntador de posición de archivo a una posición específica, en un archivo basado en la posición inicial del punto de búsqueda en el archivo. La búsqueda puede iniciarse a partir de una de tres posiciones —**SEEK_SET** inicia a partir del principio del archivo, **SEEK_CUR** inicia a partir de la posición actual en el archivo y **SEEK_END** inicia a partir del fin del archivo.
- La función **fread** lee un bloque (número específico de bytes) de datos de un archivo.

Terminología

modo de archivo abierto a	orden alfa
modo de archivo abierto a+	dígito binario
campo alfabético	bit
campo alfanumérico	byte

carácter
 campo de carácter
 conjunto de caracteres
 cerrar un archivo
 jerarquía de datos
 base de datos
 sistema de administración de base de datos
 dígito decimal
 desplazamiento
 número de doble precisión
 fin de archivo
 indicador de fin de archivo
fclose
feof
fgetc
fgets
 campo
 archivo
 almacenamiento temporal de archivo
 nombre de archivo
 modo de archivo abierto
 apuntador de archivo
 apuntador de posición de archivo
 estructura **FILE**
fopen
 entrada/salida con formato
fprintf
fputc
fputs
fread

fscanf
fseek
fwrite
 número entero
 espacios a la izquierda
 letra
 campo numérico
 desplazamiento
 abrir un archivo
 modo de archivo abierto **r**
 acceso directo
 archivo de acceso directo
 registro
 registro clave
 parámetro del número de registro
rewind
 modo de archivo abierto **r+**
SEEK_CUR
SEEK_END
SEEK_SET
 archivo de acceso secuencial
 número de una sola precisión
stderr (error estándar)
stdin (entrada estándar)
stdout (salida estándar)
 flujo
 espacios a la derecha
 modo de archivo abierto **w**
 modo de archivo abierto **w+**
 ceros y unos

Errores comunes de programación

- 11.1 Abrir un archivo existente para escritura ("**w**") cuando, de hecho, el usuario desea conservar el archivo; el contenido del archivo se descartará sin advertencia.
- 11.2 Olvidar abrir un archivo antes de intentar hacer referencia a él en un programa.
- 11.3 Usar el apuntador de archivo incorrecto para referirse a un archivo.
- 11.4 Abrir para lectura un archivo no existente.
- 11.5 Abrir un archivo para lectura o escritura sin haber obtenido los derechos de acceso al archivo apropiados (esto depende del sistema operativo).
- 11.6 Abrir un archivo para escritura cuando no hay espacio disponible en disco. En la figura 11.6 se listan los modos de apertura de archivos.
- 11.7 Puede llevar a errores devastadores abrir un archivo utilizando el modo de archivo incorrecto. Por ejemplo, abrir un archivo en el modo de escribir ("**w**") cuando debería haberse abierto en modo de actualizar ("**r+**") hace que sea descartado todo el contenido del archivo.

Prácticas sanas de programación

- 11.1 Asegúrese que en un programa las llamadas a las funciones de procesamiento de archivos contienen los apuntadores de archivo correctos.

- 11.2 Cierre cada archivo en forma explícita, tan pronto sepa que el programa ya no hará otra vez referencia al archivo.
- 11.3 Abra un archivo únicamente para lectura (y no para actualizar), si el contenido del archivo no debe modificarse. Esto evitará cambios no intencionales en el contenido del archivo. Este es otro ejemplo del principio del mínimo privilegio.

Sugerencia de rendimiento

- 11.1 Cerrar un archivo puede liberar recursos que están siendo esperados por otros usuarios o programas.
- 11.2 Muchos programadores piensan erróneamente que **sizeof** es una función, y que utilizándolo se genera una sobrecarga en tiempo de ejecución de una llamada de función. No existe tal sobrecarga, porque **sizeof** es un operador en tiempo de compilación.

Sugerencia de portabilidad

- 11.1 La estructura **FILE** depende del sistema operativo (es decir, los miembros de la estructura varían de un sistema a otro, según la forma en que cada sistema maneja sus archivos).

Ejercicios de autoevaluación

- 11.1 Llene los espacios vacíos en cada uno de los siguientes:
 - a) En última instancia, todos los elementos de datos procesados en una computadora se reducen a combinaciones de _____ y _____.
 - b) El elemento de datos más pequeño que puede procesar una computadora se conoce como un _____.
 - c) Un _____ es un grupo de registros relacionados.
 - d) Dígitos, letras y símbolos especiales se conocen como _____.
 - e) Un grupo de archivos relacionados se llama una _____.
 - f) La función _____ cierra un archivo.
 - g) El enunciado _____ lee datos de un archivo en una forma similar a la forma en que **scanf** lee a partir de **stdin**.
 - h) La función _____ lee un carácter de un archivo especificado.
 - i) La función _____ lee una línea de un archivo especificado.
 - j) La función _____ abre un archivo.
 - k) La función _____ se utiliza normalmente para leer datos de un archivo en aplicaciones de acceso directo.
 - l) La función _____ recoloca el apuntador de posición de archivo a una posición específica dentro del archivo.
- 11.2 Indique cuáles de los siguientes son verdaderos y cuáles son falsos (para aquellos que son falsos, explique por qué).
 - a) La función **fscanf** no puede ser utilizada para leer datos de la entrada estándar.
 - b) El programador debe utilizar **fopen** explícitamente, para abrir los flujos de entrada estándar, salida estándar y error estándar.
 - c) Para cerrar un archivo un programa debe llamar en forma explícita a la función **fclose**.
 - d) Si el apuntador de posición de archivo apunta a una posición en un archivo secuencial distinto al principio del mismo, el archivo debe de ser cerrado y vuelto a abrir para leer a partir del principio del mismo.
 - e) La función **fprintf** puede escribir a la salida estándar.
 - f) Los datos en los archivos de acceso secuencial se actualizan siempre sin sobrescribir otros datos.

- g) No es necesario buscar en todos los registros de un archivo de acceso directo para encontrar un registro específico.
- h) Los registros en archivos de acceso directo no son de longitud uniforme.
- i) La función `fseek` puede buscar únicamente en relación con el principio de un archivo.

11.3 Escriba un solo enunciado para que se ejecuten cada uno de los siguientes. Suponga que cada uno de estos enunciados se aplica al mismo programa.

- a) Escriba un enunciado que abra el archivo `"oldmast.dat"` para lectura y asigne el apuntador de archivo regresado a `ofPtr`.
- b) Escriba un enunciado que abra el archivo `"trans.dat"` para lectura y asigne el apuntador de archivo regresado a `tfPtr`.
- c) Escriba un enunciado que abra el archivo `"newmast.dat"` para escritura (y creación) y asigne el apuntador de archivo regresado a `nfPtr`.
- d) Escriba un enunciado que lea un registro del archivo `"oldmast.dat"`. El registro está formado del entero `accountNum`, de la cadena `Name`, y del punto flotante `currentBalance`.
- e) Escriba un enunciado que lea un registro del archivo `"trans.dat"`. El registro está formado del entero `accountNum` y del punto flotante `dollarAmount`.
- f) Escriba un enunciado que escriba un registro al archivo `"newmast.dat"`. El registro está formado del entero `accountNum`, de la cadena `Name`, y del punto flotante `currentBalance`.

11.4 Encuentre el error en cada uno de los siguientes segmentos de programa. Explique cómo se puede corregir dicho error.

- a) El archivo referido por `fPtr` (`"payables.dat"`) no ha sido abierto.
`fprintf(fPtr, "%d%s%d\n", account, company, amount);`
- b) `open("receive.dat", "r+");`
- c) El siguiente enunciado debería leer un registro del archivo `"payables.dat"`. El apuntador de archivo `payPtr` se refiere a este archivo, y el apuntador de archivo `recPtr` se refiere al archivo `"receive.dat"`
`fscanf(recPtr, "%d%s%d\n", &account, company, &amount);`
- d) El archivo `"tools.dat"` debería ser abierto para añadir datos al archivo, sin descartar los datos actuales.
`if ((tfPtr = fopen("tools.dat", "w")) != NULL)`
- e) El archivo `"courses.dat"` debería ser abierto para agregar sin modificar el contenido actual del archivo.
`if ((cfPtr = fopen("courses.dat", "w+")) != NULL)`

Respuestas a los ejercicios de autoevaluación

- 11.1 a) 1s, 0s. b) Bit, c) Archivo. d) Caracteres. e) Bases de datos f) `fclose`. g) `fscanf`. h) `getc` o bien `fgetc`. i) `fgets`. j) `fopen`. k) `fread`. l) `fseek`.
- 11.2 a) Falso. La función `fscanf` sólo puede ser utilizada para leer a partir de la entrada estándar incluyendo `stdin`, el apuntador al flujo estándar de entrada, en la llamada a `fscanf`.
- b) Falso. Estos tres flujos son automáticamente abiertos por C cuando se inicia la ejecución del programa.
- c) Falso. Los archivos serán terminados cuando se termine la ejecución del programa, pero todos los archivos deberían ser cerrados en forma explícita utilizando `fclose`.
- d) Falso. La función `rewind` puede ser utilizada para volver a colocar el apuntador de posición de archivo al principio del archivo.
- e) Verdadero.

- f) Falso. En la mayor parte de los casos, los registros de archivos secuenciales no son de longitud uniforme. Por lo tanto, es posible que al actualizar un registro se cause la sobreescritura de otros datos.
- g) Verdadero.
- h) Falso. Los registros en un archivo de acceso directo normalmente son de longitud uniforme.
- i) Falso. Es posible buscar a partir del principio del archivo, a partir del fin del archivo y a partir de la posición actual en el archivo, de acuerdo con el apuntador de posición de archivo.

- 11.3
- a) `ofPtr = fopen("oldmast.dat", "r");`
 - b) `tfPtr = fopen("trans.dat", "r");`
 - c) `nfPtr = fopen("newmast.dat", "w");`
 - d) `fscanf(ofPtr, "%d%s%f", &accountNum, name, ¤tBalance);`
 - e) `fscanf(tfPtr, "%d%f", &accountNum, &dollarAmount);`
 - f) `fprintf(nfPtr, "%d%s%.2f", accountNum, name, currentBalance);`

- 11.4
- a) Error: el archivo `"payables.dat"` no ha sido abierto antes de hacer referencia a su apuntador de archivo.
Corrección: utilice `fopen` para abrir `"payables.dat"` para escritura, agregar o actualizar.
 - b) Error: la función `open` no es una función de ANSI C.
Corrección: utilice la función `fopen`.
 - c) Error: El enunciado `fscanf` utiliza el apuntador de archivo incorrecto para referirse al archivo `"payables.dat"`.
Corrección: utilice el apuntador de archivo `payPtr` para referirse a `"payables.dat"`.
 - d) Error: el contenido del archivo será descartado, porque el archivo ha sido abierto para escritura (`"w"`).
Corrección: para añadir datos al archivo, abra el archivo ya sea para actualizar (`"r+"`) o abra el archivo para agregar (`"a"`).
 - e) Error: el archivo `"courses.dat"` está abierto para actualizar en modo `"w+"`, lo que hace que se descarte el contenido actual del archivo.
Corrección: Abra el archivo en modo `"a"`.

Ejercicios

- 11.5 Llene los espacios vacíos en cada uno de los siguientes:
- a) Las computadoras almacenan grandes cantidades de datos en dispositivos de almacenamiento secundarios como son _____.
 - b) Un _____ está compuesto de varios campos.
 - c) Un campo que pudiera contener dígitos, letras y espacios se llama un campo de _____.
 - d) Para facilitar la recuperación de registros específicos a partir de un archivo, se selecciona un campo de cada registro como _____.
 - e) La gran mayoría de la información almacenada en sistemas de computación se archiva en archivos _____.
 - f) Un grupo de caracteres relacionados que contienen significados se conocen como un _____.
 - g) Los apuntadores de archivo para los tres archivos que se abren automáticamente por C al iniciarse la ejecución de un programa se llaman _____, _____, y _____.
 - h) La función _____ escribe un carácter a un archivo especificado.
 - i) La función _____ escribe una línea a un archivo especificado.
 - j) La función _____ se usa generalmente para escribir datos a un archivo de acceso directo.
 - k) La función _____ recoloca el apuntador de posición de archivo al principio del mismo.

11.6 Indique cuáles de los siguientes son verdaderos y cuáles son falsos (en el caso de los falsos, explique por qué):

- a) Las funciones impresionantes ejecutadas por las computadoras involucran esencialmente el manejo de ceros y de unos.
- b) Las personas prefieren manipular bits en vez de caracteres y campos, porque los bits son más compactos.
- c) Las personas especifican elementos de programas y de datos como caracteres; las computadoras a continuación manipulan y procesan dichos caracteres como grupos de ceros y de unos.
- d) El código postal de una persona es un ejemplo de un campo numérico.
- e) En aplicaciones de computadora, la calle de la dirección de una persona se considera generalmente un campo alfabético.
- f) Los elementos de datos procesados por una computadora forman una jerarquía de datos, en la cual los elementos de datos se hacen más grandes y más complejos conforme se avanza desde los campos a los caracteres, a los bits, etcétera.
- g) Un registro clave identifica un registro como perteneciente a un campo particular.
- h) La mayor parte de las organizaciones almacenan toda su información en un solo archivo a fin de facilitar el procesamiento en computadora.
- i) Los archivos son siempre referidos por nombre en los programas de C.
- j) Cuando un programa genera un archivo, el archivo es automáticamente conservado por la computadora para referencia futura.

11.7 El ejercicio 11.3 le solicitó al lector que escribiera una serie de enunciados sencillos. De hecho, estos enunciados forman el núcleo de un tipo importante de programas de procesamiento de archivos, es decir, un programa de cotejo de archivos. En el procesamiento comercial de datos, es común tener varios archivos en cada sistema. Por ejemplo, en un sistema de cuentas por cobrar, generalmente existe un archivo maestro que contiene información detallada relativa a cada cliente como es nombre, dirección, número telefónico, saldo actual, límite de crédito, descuentos, arreglos contractuales, y posiblemente una historia condensada de las compras más recientes y de sus pagos.

Conforme ocurren transacciones (es decir, se efectúan ventas y llegan pagos por el correo), se introducen en un archivo. Al final de cada periodo de negocios (es decir, en algunas empresas un mes, una semana en otras y en algunos casos un día) el archivo de transacciones (llamado "trans.dat" en el ejercicio 11.3) se aplica al archivo maestro (llamado "oldmast.dat" del ejercicio 11.3), actualizando así el registro de cada cuenta en compras y pagos. Después de cada una de estas corridas de actualización, el archivo maestro se vuelve a escribir como un nuevo archivo ("newmast.dat"), que entonces se utiliza al fin del siguiente periodo de negocios, para volver a empezar el siguiente proceso de actualización.

Los programas de cotejo de archivos deben enfrentarse a ciertos problemas que no existen en programas de un solo archivo. Por ejemplo, no siempre ocurrirá una coincidencia. Un cliente existente en el archivo maestro pudiera, en el periodo de negocios actual, no haber efectuado ninguna compra o ningún pago y, por lo tanto, en el archivo de transacción no aparecerá en ningún registro para este cliente. Similarmente, un cliente que sí hizo algunas compras o pagos, quizás acaba de trasladarse a esta comunidad, y la compañía todavía no habrá tenido la oportunidad de crear un registro maestro para dicho cliente.

Utilice los enunciados escritos en el ejercicio 11.3 como base para escribir un programa completo para cotejar cuentas por cobrar. Utilice el número de cuenta en cada archivo como registro clave para cotejo. Suponga que cada archivo es un archivo secuencial con registros almacenados en orden creciente de número de cuenta.

Cuando ocurra una coincidencia (es decir, cuando registros con el mismo número de cuenta aparezcan tanto en el archivo maestro como en el archivo de transacción) añada la cantidad en dólares existente en el archivo de transacción al saldo actual existente en el archivo maestro, y escriba el registro "newmast.dat". (Suponga que las adquisiciones o compras están indicadas por cantidades positivas en el archivo de transacción, y que los pagos están indicados por cantidades negativas.) Cuando exista un registro maestro relativo a una cuenta particular sin registro de transacción correspondiente, simplemente

escriba el registro maestro a "newmast.dat". Cuando exista un registro de transacción sin el correspondiente registro maestro, imprima el mensaje "Unmatched transaction record for account number ..." (incluya el número de cuenta proveniente del registro de transacción).

11.8 Después de escribir el programa del ejercicio 11.7 escriba un programa sencillo para crear alguna información de prueba para verificar el programa del ejercicio 11.7. Utilice los datos de cuentas de muestras siguientes:

Archivo maestro Número de cuenta	Nombre	Balance
100	Alan Jones	348.17
300	Mary Smith	27.19
500	Sam Sharp	0.00
700	Suzy Green	-14.22

Archivo de transacción: Número de cuenta	Cantidad en dólares
100	27.14
300	62.11
400	100.56
900	82.17

11.9 Ejecute el programa del ejercicio 11.7 utilizando los archivos de los datos de prueba creados en el ejercicio 11.8 Utilice el programa enlistado de la sección 11.7 para imprimir un nuevo archivo maestro. Verifique cuidadosamente los resultados.

11.10 Es posible (y de hecho común) tener varios registros de transacciones para un mismo registro clave. Esto ocurre porque un cliente en particular durante un periodo de negocios pudiera efectuar varias compras y varios pagos. Vuelva a escribir su programa de cotejo de cuentas por cobrar del ejercicio 11.7 para que incluya la posibilidad de manejar varios registros de transacción con el mismo registro clave. Modifique los datos de prueba del ejercicio 11.8 para incluir los siguientes registros de transacción adicionales:

Número de cuenta	Cantidad en dólares
300	83.89
700	80.78
700	1.53

11.11 Escriba enunciados que lleven a cabo cada uno de los siguientes. Suponga que la estructura

```
struct person {
    char lastName[15];
    char firstName[15];
    char age[2];
}
```

ha sido definida, y que el archivo ya está abierto para escritura.

- a) Inicialice el archivo "nameage.dat" de tal forma que existan 100 registros con lastName = "unassigned", firstName = "", y age = "0"
- b) Introduzca 10 apellidos, nombres y edades y escribalos al archivo.
- c) Actualice un registro; si no existe información en el registro, indique al usuario "No info".
- d) Borre un registro que tenga información mediante la reinicialización de dicho registro en particular.

11.12 Usted es el propietario de una ferretería y necesita mantener un inventario que le pueda indicar cuáles son las herramientas que tiene, cuántas tiene y el costo de cada una. Escriba un programa que inicialice el archivo "hardware.dat" a 100 registros vacíos, que le permita introducir los datos correspondientes a cada herramienta, enliste todas sus herramientas, borrar un registro correspondiente a una herramienta que ya no posea, y le deje actualizar cualquier información dentro del archivo. El número de identificación de la herramienta deberá ser el número de registro. Utilice la siguiente información para iniciar su archivo:

Registro #	Nombre de la herramienta	Cantidad	Costo
3	Electric sander	7	57.98
17	Hammer	76	11.99
24	Jig saw	21	11.00
39	Lawn mower	3	79.50
56	Power saw	18	99.99
68	Screwdriver	106	6.99
77	Sledge hammer	11	21.50
83	Wrench	34	7.50

11.13 Generador de palabras de números telefónicos. Los marcadores telefónicos estándar contienen los dígitos 0 al 9. Los números 2 hasta el 9 cada uno de ellos tiene tres letras asociadas, tal y como se indican en la tabla siguiente:

Dígito	Letras
2	A B C
3	D E F
4	G H I
5	J K L
6	M N O
7	P R S
8	T U V
9	W X Y

Muchas personas encuentran difícil memorizar los números telefónicos, por lo que utilizan la correspondencia existente entre dígitos y letras para desarrollar palabras de siete letras, que correspondan a sus números telefónicos. Por ejemplo, una persona cuyo número telefónico es 686-2377 pudiera utilizar la correspondencia indicada en la tabla precedente, para desarrollar la palabra de siete letras "NUMBERS".

Los negocios frecuentemente intentan obtener números telefónicos que sean fáciles de recordar para sus clientes. Si un negocio puede anunciar una sola palabra para que la marquen sus clientes, entonces sin duda dicho negocio recibirá unas cuantas llamadas más.

Cada palabra de siete letras corresponde a exactamente un número telefónico de siete dígitos. El restaurante que desee aumentar su negocio de venta de comidas para llevar, podría seguramente hacerlo con el número 825-3688 (es decir "TAKEOUT").

Cada número telefónico de 7 dígitos corresponde a muchas palabras distintas de 7 letras. Desafortunadamente, la mayor parte de ellas representan yuxtaposiciones irreconocibles de letras. Es posible, sin embargo, que el propietario de una peluquería se sintiera complacido al saber que el número telefónico de su tienda, 424-7288 corresponde a "HAIRCUT". El propietario de una tienda de licores estaría encantado, sin duda, al averiguar que el número telefónico de la tienda 233-7226, corresponde a "BEERCAN". Un médico veterinario con el número telefónico 738-2273 estaría complacido al aprender que el número corresponde a las letras "PETCARE".

Escriba un programa en C que, dado un número de 7 dígitos, escriba a un archivo todas las palabras posibles de 7 letras que correspondan a dicho número. Existen 2187 (3 a la séptima potencia) palabras posibles. Evite números telefónicos que contengan los dígitos 0 y 1.

11.14 Si tiene disponible un diccionario computarizado, modifique el programa que escribió en el Ejercicio 11.13 para buscar las palabras en el diccionario. Algunas combinaciones de siete letras creadas por este programa consisten de dos o más palabras (el número telefónico 843-2677 produce "THEBOSS").

11.15 Modifique el ejemplo de la figura 8.14 para utilizar las funciones fgetc y fputs, en vez de getchar y puts. El programa deberá darle al usuario la opción de leer desde la entrada estándar y escribir a la salida estándar, o de leer de un archivo especificado y escribir a un archivo especificado. Si el usuario se decide por la segunda opción, haga que el usuario introduzca los nombres de archivo correspondientes a los archivos de entrada y de salida.

11.16 Escriba un programa que utilice el operador sizeof para determinar los tamaños en bytes de varios tipos de datos en su sistema de computación. Escriba los resultados al archivo "datasize.dat" de tal forma que más tarde se puedan imprimir los resultados. El formato para los resultados del archivo deberá ser:

Data type	Size
char	1
unsigned char	1
short int	2
unsigned short int	2
int	4
unsigned int	4
long int	4
unsigned long int	4
float	4
double	8
long double	16

Nota: los tamaños de tipo en su computadora pudieran no ser iguales a los arriba listados.

11.17 En el ejercicio 7.19, usted escribió una simulación en software de una computadora, que utilizaba un lenguaje máquina especial llamado lenguaje de máquina Simpletron (LMS). En la simulación, cada vez que usted deseaba ejecutar un programa LMS, usted introducía el programa en el simulador a partir del teclado. Si al capturar el programa LMS cometía algún error, el simulador se volvía a iniciar y el código LMS se volvía a introducir. Sería agradable, en vez de tener que escribirlo cada vez, poder leer el programa LMS a partir de un archivo. Esto reduciría tiempo y errores en la preparación de la ejecución de programas LMS.

- a) Modifique el simulador que escribió en el ejercicio 7.19 para leer programas LMS a partir de un archivo especificado por el usuario desde el teclado.
- b) Una vez que Simpletron se ejecuta, saca el contenido de sus registros y de memoria a la pantalla. Sería también deseable capturar la salida en un archivo, por lo que modifique el simulador para escribir su salida a un archivo, además de que despliegue dicha salida en la pantalla.

12

Estructuras de datos

Objetivos

- Ser capaz de asignar y liberar memoria dinámicamente para objetos de datos.
- Ser capaz de formar estructuras de datos enlazadas mediante el uso de apuntadores, estructuras autorreferenciadas y recursión.
- Ser capaz de crear y manipular listas enlazadas, colas, pilas y árboles binarios.
- Comprender varias aplicaciones importantes de las estructuras de datos enlazadas.

*Mucho de lo que sujeto, no puedo liberar;
Mucho que puedo liberar regresó a mí.*

Lee Wilson Dodd

*'¿Quieres andar un poco más rápido?' dijo una merluza
a un caracol. 'Hay una tortuga detrás de nosotros,
y me está pisando la cola'.*

Lewis Carroll

Siempre hay lugar en la cima.

Daniel Webster

Empuja —mantente en movimiento.

Thomas Morton

Pienso que no veré jamás

Tan bello poema como es un árbol.

Joyce Kilmer

Sinopsis

- 12.1 Introducción
- 12.2 Estructuras autorreferenciadas
- 12.3 Asignación dinámica de memoria
- 12.4 Listas enlazadas
- 12.5 Pilas
- 12.6 Colas de espera
- 12.7 Árboles

Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencias de rendimiento • Sugerencias de portabilidad • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.

12.1 Introducción

Hemos estudiado *estructuras de datos* de tamaño fijo, como arreglos de un solo subíndice, de doble subíndice y **structs**. Este capítulo presenta *estructuras dinámicas de datos*, cuyo tamaño crece y se encoge en tiempo de ejecución. Las *listas enlazadas* son colecciones de elementos de datos “alineados en una fila” —en una lista enlazada las inserciones y las eliminaciones se efectúan en cualquier parte. Las *pilas* son importantes en compiladores y sistemas operativos —en una pila las inserciones y las eliminaciones se efectúan únicamente en un extremo— en su *parte superior*. Las *colas de espera* representan líneas de espera; las inserciones se efectúan en la parte trasera (también conocida como la *cola*) de la misma, y las eliminaciones se hacen de la parte delantera (también conocida como cabeza de la cola). Los *árboles binarios* facilitan la búsqueda y clasificación de los datos a alta velocidad, la eliminación eficiente de elementos duplicados de datos, la representación de sistemas de directorios de archivo y las expresiones de compilación en lenguaje de máquina. Cada una de esas estructuras de datos tiene muchas otras interesantes aplicaciones.

Analizaremos cada uno de los tipos principales de estructuras de datos y pondremos en operación programas que crean y manipulan estas estructuras de datos. En la siguiente parte del libro —la introducción a C++ y a la programación orientada a objetos, en los capítulos del 15 hasta el 21 —estudiaremos la abstracción de datos. Esta técnica nos permitirá construir estas estructuras de datos de una forma dramáticamente diferente, diseñada para producir software mucho más fácil de mantener y especialmente mucho más fácil de volver a utilizar.

Este es un capítulo retador. Los programas son sustanciales e incorporan la mayor parte de lo que ha aprendido en los capítulos anteriores. Estos programas están llenos de manipulaciones de apuntadores, un tema que muchas personas consideran de entre los temas más difíciles de C. El capítulo está lleno de programas altamente prácticos, que podrá utilizar en cursos más avanzados; el capítulo incluye una valiosa colección de ejercicios que enfatizan aplicaciones prácticas de las estructuras de datos.

Sinceramente esperamos que usted intentará el importante proyecto descrito en la sección especial titulada “Cómo construir su propio compilador”. Usted ha estado utilizando un compilador para traducir sus programas C a lenguaje de máquina, a fin de poder ejecutar sus programas en su computadora. En este proyecto, realmente construirá su propio compilador. Este leerá un archivo de enunciados, escritos en un simple, aunque poderoso lenguaje de alto nivel, similar a las versiones primeras del popular lenguaje BASIC. Su compilador convertirá estos enunciados en un archivo de instrucciones de lenguaje de máquina Simpletron. LMS es el lenguaje que aprendió en la sección especial del capítulo 7, “Cómo construir su propia computadora”. ¡A continuación su programa simulador Simpletron ejecutará el programa LMS producido por su compilador! Este proyecto le dará la maravillosa oportunidad de practicar la mayor parte de lo que en este curso ha aprendido. La sección especial lo lleva cuidadosamente a través de las especificaciones del lenguaje de alto nivel, y describe los algoritmos que necesitará para convertir cada tipo de enunciado del lenguaje de alto nivel a instrucciones en lenguaje máquina. Si usted es afecto a aceptar retos, pudiera intentar llevar a cabo las muchas mejoras, tanto al compilador como al simulador Simpletron que se sugieren en los ejercicios.

12.2 Estructuras autorreferenciadas

Una *estructura autorreferenciada* contiene un miembro de apuntador que apunta a una estructura del mismo tipo de estructura. Por ejemplo, la definición

```
struct node {
    int data;
    struct node *nextPtr;
};
```

define un tipo, **struct node**. Una estructura del tipo **struct node** tiene dos miembros —el miembro entero **data** y el miembro de apuntador **nextPtr**. El miembro **nextPtr** apunta a una estructura de tipo **struct node** —una estructura del mismo tipo que la que se está declarando aquí, de ahí el “término estructura autorreferenciada”. El miembro **nextPtr** se conoce como un *enlace o vínculo* —es decir, **nextPtr** puede ser utilizada para “vincular” una estructura del tipo **struct node** con otra estructura del mismo tipo. Las estructuras autorreferenciadas pueden ser enlazadas juntas para formar útiles estructuras de datos como son las listas, las colas de espera, las pilas y los árboles. En la figura 12.1 se ilustran dos estructuras autorreferenciadas, enlazadas juntas para formar una lista. Note que se coloca una diagonal —que representa un apuntador **NULL**— en el miembro enlazado de la segunda estructura autorreferenciada, para indicar que el enlace no apunta a otra estructura. La diagonal aparece sólo para fines de ilustración; no corresponde al carácter de diagonal de C. Igual que el carácter **NULL** indica el final de una cadena, normalmente un apuntador **NULL** indica el fin de una estructura de datos.

Error común de programación 12.1

No establecer a **NULL** el enlace en el último nodo de una lista.

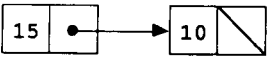


Fig. 12.1 Dos estructuras autorreferenciadas enlazadas juntas.

12.3 Asignación dinámica de memoria

La creación y mantenimiento de estructuras dinámicas de datos, requiere de la *asignación dinámica de memoria* —la capacidad por parte de un programa de obtener, en tiempo de ejecución, más espacio de memoria para contener nuevos nodos, y de poder liberar espacio ya no requerido. El límite de la asignación dinámica de memoria puede ser tan grande como la totalidad de memoria física disponible en la computadora, o la cantidad de memoria virtual disponible, en un sistema de memoria virtual. A menudo, los límites son mucho menores, porque la memoria disponible deberá ser compartida entre muchos usuarios.

Las funciones `malloc` y `free` y el operador `sizeof`, son esenciales a la asignación dinámica de memoria. La función `malloc` toma como argumento el número de bytes a asignarse, y regresa un apuntador del tipo `void*` (*apuntador a void*) a la memoria asignada. Un apuntador `void*` puede asignarse a una variable de cualquier tipo de apuntador. Normalmente la función `malloc` se utiliza conjuntamente con el operador `sizeof`. Por ejemplo, el enunciado

```
newPtr = malloc(sizeof(struct node));
```

evalúa `sizeof(struct node)` para determinar el tamaño en bytes de una estructura del tipo `struct node`, asigna en memoria una nueva área de tamaño `sizeof(struct node)` bytes, y almacena en la variable `newPtr` un apuntador a la memoria asignada. Si no existe memoria disponible, `malloc` regresa un apuntador `NULL`.

La función `free` cancela la asignación de la memoria —es decir, se regresa la memoria al sistema, de tal forma que en el futuro ésta pueda ser vuelta a asignar. Para liberar memoria asignada dinámicamente mediante una llamada `malloc` previa, utilice el enunciado

```
free(newPtr);
```

En las siguientes secciones se analizan listas, pilas, colas de espera y árboles. Cada una de estas estructuras de datos se crea y se mantiene utilizando la asignación dinámica de memoria y las estructuras autorreferenciadas.

Sugerencia de portabilidad 12.1

El tamaño de una estructura no es necesariamente la suma de los tamaños de sus miembros. Esto es debido a varios requisitos de alineación de límites, que son dependientes de la máquina (vea el capítulo 10).

Error común de programación 12.2

Suponer que el tamaño de una estructura es simplemente la suma de los tamaños de sus miembros.

Práctica sana de programación 12.1

Para determinar el tamaño de una estructura utilice el operador `sizeof`.

Práctica sana de programación 12.2

Al utilizar `malloc`, compruebe si es `NULL` el valor de regreso de apuntador. Si la memoria solicitada no ha sido asignada imprima un mensaje de error.

Error común de programación 12.3

No regresar memoria dinámicamente asignada cuando ésta ya no es necesaria, puede hacer que el sistema se quede sin memoria prematuramente. Esto se conoce a veces como “fuga de memoria”.

Práctica sana de programación 12.3

Cuando ya no se requiera memoria que fue dinámicamente asignada, utilice `free`, para regresar esta memoria inmediatamente al sistema.

Error común de programación 12.4

Utilizando `malloc`, liberar memoria no dinámicamente asignada.

Error común de programación 12.5

Referirse a memoria que ya ha sido liberada.

12.4 Listas enlazadas

Una *lista enlazada* es una colección lineal de estructuras autorreferenciadas llamadas *nodos*, conectadas por *enlaces* de apuntador —de ahí el término lista “enlazada”. Se tiene acceso a una lista enlazada vía un apuntador al primer nodo de la lista. Se puede tener acceso a los nodos subsecuentes vía el apuntador de enlace almacenado en cada nodo. Por regla convencional, para marcar el fin de la lista, el apuntador de enlace, en el último nodo de una lista, se define a `NULL`. En una lista enlazada los datos se almacenan dinámicamente —cada nodo se crea conforme sea necesario. Un nodo puede contener datos de cualquier tipo, incluyendo otras `struct`. Las pilas y las colas de espera también son estructuras lineales de datos, y como veremos, son versiones restringidas de listas enlazadas. Los árboles son estructuras no lineales de datos.

Las listas de datos pueden ser almacenadas en arreglos, pero las listas enlazadas proporcionan varias ventajas. Una lista enlazada es apropiada cuando no es predecible de inmediato el número de elementos de datos a representarse en la estructura. Las listas enlazadas son dinámicas, por lo que conforme sea necesario la longitud de una lista puede aumentar o disminuir. Por su parte, el tamaño de un arreglo no puede ser modificado, porque la memoria del arreglo es asignada en tiempo de compilación. Los arreglos pueden llenarse. Las listas enlazadas sólo se llenan cuando el sistema no tiene suficiente memoria para satisfacer las solicitudes de asignación dinámica de almacenamiento.

Sugerencia de rendimiento 12.1

Podría declararse un arreglo que contenga más elementos que el número esperado de elementos de datos, pero esto puede desperdiciar memoria. En estas situaciones las listas enlazadas pueden obtener una mejor utilización de la memoria.

Las listas enlazadas pueden mantenerse en orden, insertando cada elemento nuevo en el punto apropiado dentro de la lista.

Sugerencia de rendimiento 12.2

Puede resultar muy tardado insertar y eliminar en un arreglo ya ordenado —deberán ser desplazados en forma apropiada. Todos los elementos que sigan al elemento insertado o borrado.

Sugerencia de rendimiento 12.3

Los elementos de un arreglo se almacenan en forma contigua en memoria. Esto permite acceso inmediato a cualquier arreglo del elemento, porque la dirección de cualquier elemento puede ser calculada directamente, basada en su posición en relación con el principio del arreglo. Las listas enlazadas no proporcionan un acceso inmediato como éste a sus elementos.

Normalmente, los nodos de las listas enlazadas no están almacenados en memoria en forma contigua. Sin embargo, lógicamente, los nodos de una lista enlazada aparecen como contiguos. En la figura 12.2 se ilustra una lista enlazada con varios nodos.

Sugerencia de rendimiento 12.4

Tratándose de estructuras de datos que crecen o se reducen en tiempo de ejecución, es posible ahorrar memoria utilizando asignación dinámica de memoria (en vez de los arreglos). Recuerde, sin embargo, que los apuntadores toman espacio, y que la asignación dinámica de memoria incurre en sobrecarga por las llamadas de función.

El programa de la figura 12.3 (cuya salida se muestra en la figura 12.4) manipula una lista de caracteres. El programa da dos opciones: 1) insertar un carácter en la lista en orden alfabético (función **insert**) y 2) borrar un carácter de la lista (función **delete**). Este es un programa grande y complejo. Sigue un análisis detallado respecto al programa. El ejercicio 12.20 solicita al estudiante que ponga en operación una función recursiva, que imprima una lista al revés. En el ejercicio 12.21 se le pide al estudiante que ponga en marcha una función recursiva, que busque en una lista enlazada un elemento particular de datos.

Las dos funciones primarias de las listas enlazadas son **insert** y **delete**. La función **isEmpty** se conoce como una *función predicada* —no altera en forma alguna la lista; más bien determina si la lista está vacía (es decir, si el apuntador al primer nodo de la lista es **NULL**). Si la lista está vacía, se regresa 1; de lo contrario se regresa 0. La función **printList** imprime la lista.

En la lista los caracteres se insertan en orden alfabético. La función **insert** recibe la *dirección* de la lista y el carácter a insertarse. Es necesaria la dirección de la lista cuando se va a insertar un valor en el inicio de la lista. Proporcionar la dirección de la lista permite que se pueda modificar la lista (el apuntador al primer nodo de la lista) vía una llamada por referencia. Dado que la lista propiamente dicha es un apuntador (a su primer elemento), pasar la dirección de la lista crea un *apuntador a un apuntador* (es decir, una *doble indirección*). Este es un concepto complejo y requiere de cuidadosa programación. Los pasos para la inserción de un carácter en la lista son como sigue (vea la figura 12.5):

- 1) Crear un nodo llamando **malloc**, asignando a **newPtr** la dirección de la memoria asignada, asignando el carácter a insertarse a **newPtr->data**, y asignando **NULL** a **newPtr->nextPtr**.
- 2) Inicialice **previousPtr** a **NULL**, y **currentPtr** a ***sPtr** (el apuntador al inicio de la lista). Los apuntadores **previousPtr** y **currentPtr** se utilizan para almacenar las posiciones del nodo anterior al punto de inserción y del nodo posterior al punto de inserción.

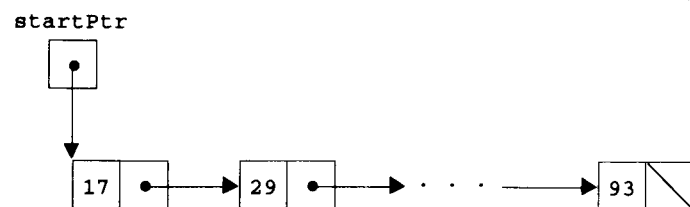


Fig. 12.2 Representación gráfica de una lista enlazada.

```

/* Operating and maintaining a list */
#include <stdio.h>
#include <stdlib.h>

struct listNode { /* self-referential structure */
    char data;
    struct listNode *nextPtr;
};

typedef struct listNode LISTNODE;
typedef LISTNODE *LISTNODEPTR;

void insert(LISTNODEPTR *, char);
char delete(LISTNODEPTR *, char);
int isEmpty(LISTNODEPTR);
void printList(LISTNODEPTR);
void instructions(void);

main()
{
    LISTNODEPTR startPtr = NULL;
    int choice;
    char item;

    instructions(); /* display the menu */
    printf("? ");
    scanf("%d", &choice);

    while (choice != 3) {

        switch (choice) {
            case 1:
                printf("Enter a character: ");
                scanf("%c", &item);
                insert(&startPtr, item);
                printList(startPtr);
                break;
            case 2:
                if (!isEmpty(startPtr)) {
                    printf("Enter character to be deleted: ");
                    scanf("%c", &item);

                    if (delete(&startPtr, item)) {
                        printf("%c deleted.\n", item);
                        printList(startPtr);
                    }
                    else
                        printf("%c not found.\n\n", item);
                }
                else
                    printf("List is empty.\n\n");

                break;
        }
    }
}

```

Fig. 12.3 Cómo insertar y borrar nodos en una lista (parte 1 de 3).

```

        default:
            printf("Invalid choice.\n\n");
            instructions();
            break;
    }

    printf("? ");
    scanf("%d", &choice);
}

printf("End of run.\n");
return 0;
}

/* Print the instructions */
void instructions(void)
{
    printf("Enter your choice:\n"
           "  1 to insert an element into the list.\n"
           "  2 to delete an element from the list.\n"
           "  3 to end.\n");
}

/* Insert a new value into the list in sorted order */
void insert(LISTNODEPTR *sPtr, char value)
{
    LISTNODEPTR newPtr, previousPtr, currentPtr;

    newPtr = malloc(sizeof(LISTNODE));

    if (newPtr != NULL) { /* is space available */
        newPtr->data = value;
        newPtr->nextPtr = NULL;

        previousPtr = NULL;
        currentPtr = *sPtr;

        while (currentPtr != NULL && value > currentPtr->data) {
            previousPtr = currentPtr; /* walk to ... */
            currentPtr = currentPtr->nextPtr; /* ... next node */
        }

        if (previousPtr == NULL) {
            newPtr->nextPtr = *sPtr;
            *sPtr = newPtr;
        }
        else {
            previousPtr->nextPtr = newPtr;
            newPtr->nextPtr = currentPtr;
        }
    }
    else
        printf("%c not inserted. No memory available.\n", value);
}

```

Fig. 12.3 Cómo insertar y borrar nodos en una lista (parte 2 de 3).

```

/* Delete a list element */
char delete(LISTNODEPTR *sPtr, char value)
{
    LISTNODEPTR previousPtr, currentPtr, tempPtr;

    if (value == (*sPtr)->data) {
        tempPtr = *sPtr;
        *sPtr = (*sPtr)->nextPtr; /* de-thread the node */
        free(tempPtr); /* free the de-threaded node */
        return value;
    }
    else {
        previousPtr = *sPtr;
        currentPtr = (*sPtr)->nextPtr;

        while (currentPtr != NULL && currentPtr->data != value) {
            previousPtr = currentPtr; /* walk to ... */
            currentPtr = currentPtr->nextPtr; /* ... next node */
        }

        if (currentPtr != NULL) {
            tempPtr = currentPtr;
            previousPtr->nextPtr = currentPtr->nextPtr;
            free(tempPtr);
            return value;
        }
    }

    return '\0';
}

/* Return 1 if the list is empty, 0 otherwise */
int isEmpty(LISTNODEPTR sPtr)
{
    return sPtr == NULL;
}

/* Print the list */
void printList(LISTNODEPTR currentPtr)
{
    if (currentPtr == NULL)
        printf("List is empty.\n\n");
    else {
        printf("The list is:\n");

        while (currentPtr != NULL) {
            printf("%c --> ", currentPtr->data);
            currentPtr = currentPtr->nextPtr;
        }

        printf("NULL\n\n");
    }
}

```

Fig. 12.3 Cómo insertar y borrar nodos en una lista (parte 3 de 3)

```
Enter your choice:
  1 to insert an element into the list.
  2 to delete an element from the list.
  3 to end.
? 1
Enter a character: B
The list is:
B -> NULL

? 1
Enter a character: A
The list is:
A -> B -> NULL

? 1
Enter a character: C
The list is:
A -> B -> C -> NULL

? 2
Enter character to be deleted: D
D not found.

? 2
Enter character to be deleted: B
B deleted.
The list is:
A -> C -> NULL

? 2
Enter character to be deleted: C
C deleted.
The list is:
A -> NULL

? 2
Enter character to be deleted: A
A deleted.
List is empty.

? 4
Invalid choice.

Enter your choice:
  1 to insert an element into the list.
  2 to delete an element from the list.
  3 to end.
? 3
End of run.
```

Fig. 12.4 Salida de muestra del programa de la figura 12.3.

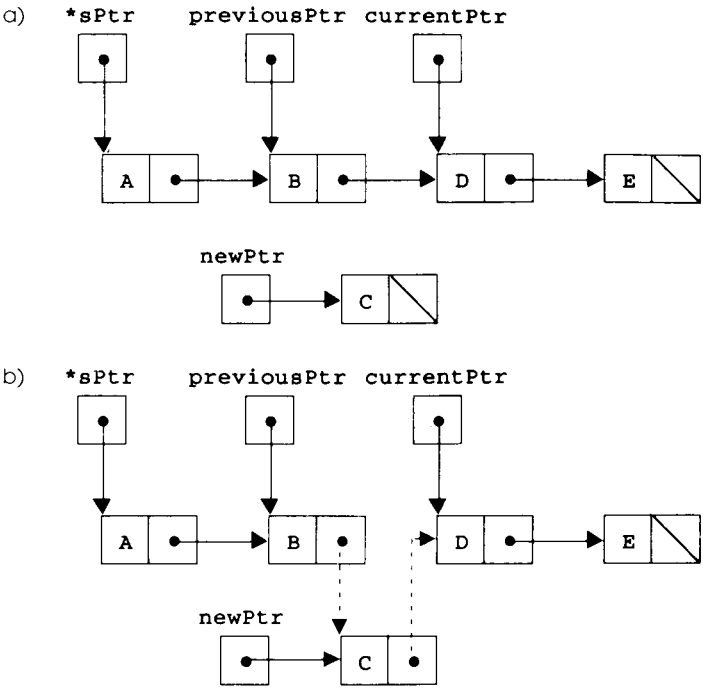


Fig. 12.5 Cómo insertar un nodo en orden dentro de una lista.

- 3) En tanto `currentPtr` no sea `NULL` y el valor a insertarse sea mayor que `currentPtr->data`, asigne `currentPtr` a `previousPtr` y avance `currentPtr` al siguiente nodo en la lista. Esto posiciona en la lista el punto de inserción del valor.
- 4) Si `previousPtr` es `NULL`, se inserta el nuevo nodo como primer nodo de la lista. Asigne `*sPtr` a `newPtr->nextPtr` (el nuevo enlace de nodo apunta al anterior primer nodo), y asigne `newPtr` a `*sPtr` (`*sPtr` apunta al nuevo nodo). Si `previousPtr` no es `NULL`, el nuevo nodo se inserta en su lugar. Asigne `newPtr` a `previousPtr->nextPtr` (el nodo anterior apunta al nuevo nodo), y asigne `currentPtr` a `newPtr->nextPtr` (el enlace de nuevo nodo apunta al nodo actual).

Práctica sana de programación 12.4

Asigne `NULL` al miembro de enlace de un nuevo nodo. Los apuntadores deben ser inicializados antes de ser utilizados.

En la figura 12.5 se ilustra la inserción de un nodo conteniendo el carácter 'C' en una lista ordenada. La parte a) de la figura muestra la lista y el nuevo nodo antes de la inserción. La parte b) de la figura muestra el resultado de la inserción del nuevo nodo. Los apuntadores reasignados están representados por flechas y líneas punteadas.

La función `delete` recibe la dirección del apuntador hacia el principio de la lista y un carácter a borrarse. Los pasos para borrar un carácter de la lista son como siguen:

- 1) Si el carácter a borrarse coincide con el primer carácter del primer nodo de la lista, asigna `*sPtr` a `tempPtr` (`tempPtr` será utilizado para liberar, usando `free`, la memoria no necesaria), asigna `(*sPtr) -> nextPtr` a `*sPtr` (`*sPtr` ahora apunta al segundo nodo de la lista), `free` libera la memoria apuntada por `tempPtr`, y regresa el carácter que fue borrado.
- 2) De no ser así, inicializa `previousPtr` con `*sPtr` e inicializa `currentPtr` con `(*sPtr) -> nextPtr`.
- 3) En tanto `currentPtr` no sea `NULL` y el valor a borrarse no sea igual `currentPtr -> data`, asigna `currentPtr` a `previousPtr`, y asigna `currentPtr -> nextPtr` a `currentPtr`. Esto localizará el carácter a borrarse, si está contenido dentro de la lista.
- 4) Si `currentPtr` no es `NULL`, asigna `currentPtr` a `tempPtr`, asigna `currentPtr -> nextPtr` a `previousPtr -> nextPtr`, libera el nodo al cual apunta `tempPtr`, y regresa el carácter que fue borrado de la lista. Si `currentPtr` es `NULL`, regresa el carácter `NULL` (`'\0'`), para significar que el carácter a borrarse no fue encontrado dentro de la lista.

En la figura 12.6 se ilustra el borrado de un nodo de una lista enlazada. La parte a) de la figura muestra la lista enlazada, antes de la operación de inserción anterior. La parte b) muestra la reasignación del elemento de enlace de `previousPtr` y la asignación de `currentPtr` a `tempPtr`. El apuntador `tempPtr` se utiliza para liberar la memoria asignada para almacenar 'C'.

La función `printList` recibe un apuntador al inicio de la lista como un argumento, y se refiere al apuntador como `currentPtr`. La función primero determina si la lista está vacía. Si es así, `printList` imprime "The list is empty" y termina. De lo contrario, imprime

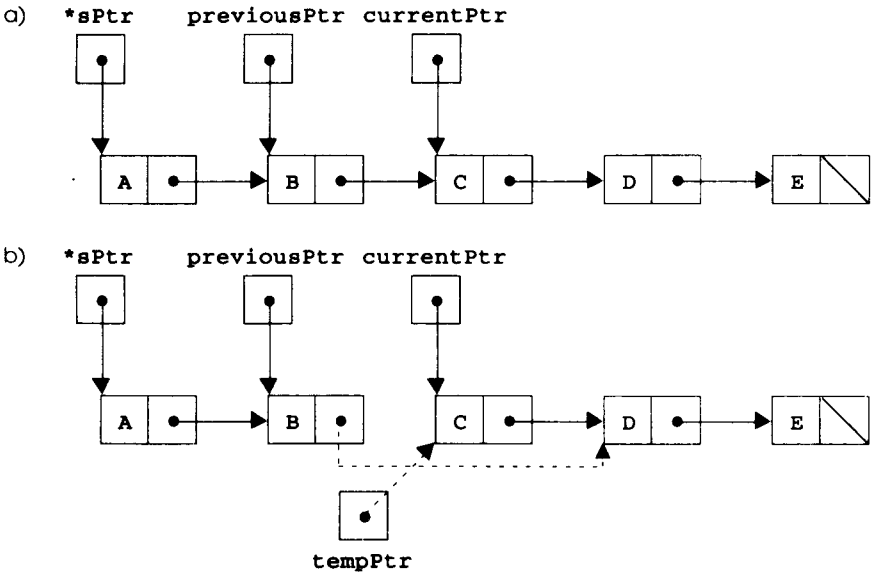


Fig. 12.6 Cómo borrar un nodo de una lista.

los datos en la lista. En tanto `currentPtr` no sea `NULL`, `currentPtr -> data` será impreso por la función, y `currentPtr -> nextPtr` será asignado a `currentPtr`. Note que si en el último nodo el enlace de la lista no es `NULL`, el algoritmo de impresión tratará de imprimir más allá del final de la lista, y ocurrirá un error. El algoritmo de impresión es idéntico para listas enlazadas, pilas y colas de espera.

12.5 Pilas

Una *pila* es una versión restringida de una lista enlazada. A una pila se le pueden añadir y retirar nuevos nodos únicamente de su parte superior. Por esta razón, se conoce una pila como una estructura de datos como *últimas entradas, primeras salidas* (*LIFO por last-in, first-out*). Se referencia una pila mediante un apuntador al elemento superior de la misma. El miembro de enlace en el último nodo de la pila se define a `NULL`, para indicar que se trata de la parte inferior de la pila misma.

En la figura 12.7 se ilustra una pila con varios nodos. Note que las pilas y las listas enlazadas se representan en forma idéntica. La diferencia entre las pilas y las listas enlazadas es que en una lista enlazada las inserciones y borrados pueden ocurrir en cualquier parte, pero en una pila únicamente en su parte superior.

Error común de programación 12.6

No definir a `NULL` el enlace en el nodo inferior de una pila.

Las funciones primarias utilizadas para manipular una pila son `push` y `pop`. La función `push` crea un nuevo nodo y lo coloca en la parte superior de la pila. La función `pop` elimina un nodo de la parte superior de la pila, liberando la memoria que fue asignada al nodo retirado, y regresando el valor retirado.

El programa de la figura 12.8 (cuya salida se muestra en la salida 12.9) representa una pila de enteros simple. El programa presenta tres opciones: 1) incluir (`push`) un valor en la pila (función `push`), 2) retirar (`pop`) un valor de la pila (función `pop`) y 3) terminar el programa.

La función `push` coloca un nuevo nodo en la parte superior de la pila. La función está formada de tres pasos:

- 1) Crear un nuevo nodo llamando a `malloc`, asignando la posición de la memoria asignada a `newPtr`, asignando el valor a colocarse en la pila a `newPtr -> data`, y asignar `NULL` a `newPtr -> nextPtr`.
- 2) Asignar `*topPtr` (el apuntador a la parte superior de la pila) a `newPtr -> nextPtr` —el miembro de enlace de `newPtr` ahora apunta al nodo superior anterior.
- 3) Asigna `newPtr` a `*topPtr` —`*topPtr` apunta ahora a la nueva parte superior de la pila.

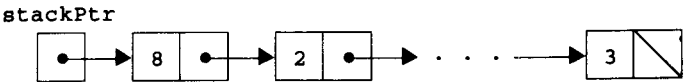


Fig. 12.7 Representación gráfica de una pila.

```

/* dynamic stack program */
#include <stdio.h>
#include <stdlib.h>

struct stackNode { /* self-referential structure */
    int data;
    struct stackNode *nextPtr;
};

typedef struct stackNode STACKNODE;
typedef STACKNODE *STACKNODEPTR;

void push(STACKNODEPTR *, int);
int pop(STACKNODEPTR *);
int isEmpty(STACKNODEPTR);
void printStack(STACKNODEPTR);
void instructions(void);

main()
{
    STACKNODEPTR stackPtr = NULL; /* points to the stack top */
    int choice, value;

    instructions();
    printf("? ");
    scanf("%d", &choice);

    while (choice != 3) {
        switch (choice) {
            case 1: /* push value onto stack */
                printf("Enter an integer: ");
                scanf("%d", &value);
                push(&stackPtr, value);
                printStack(stackPtr);
                break;
            case 2: /* pop value off stack */
                if (!isEmpty(stackPtr))
                    printf("The popped value is %d.\n",
                        pop(&stackPtr));

                printStack(stackPtr);
                break;
            default:
                printf("Invalid choice.\n\n");
                instructions();
                break;
        }

        printf("? ");
        scanf("%d", &choice);
    }

    printf("End of run.\n");
    return 0;
}

```

Fig. 12.8 Un programa de pilas simple (parte 1 de 3).

```

/* Print the instructions */
void instructions(void)
{
    printf("Enter choice:\n"
        "1 to push a value on the stack\n"
        "2 to pop a value off the stack\n"
        "3 to end program\n");
}

/* Insert a node at the stack top */
void push(STACKNODEPTR *topPtr, int info)
{
    STACKNODEPTR newPtr;

    newPtr = malloc(sizeof(STACKNODE));
    if (newPtr != NULL) {
        newPtr->data = info;
        newPtr->nextPtr = *topPtr;
        *topPtr = newPtr;
    }
    else
        printf("%d not inserted. No memory available.\n", info);
}

/* Remove a node from the stack top */
int pop(STACKNODEPTR *topPtr)
{
    STACKNODEPTR tempPtr;
    int popValue;

    tempPtr = *topPtr;
    popValue = (*topPtr)->data;
    *topPtr = (*topPtr)->nextPtr;
    free(tempPtr);
    return popValue;
}

/* Print the stack */
void printStack(STACKNODEPTR currentPtr)
{
    if (currentPtr == NULL)
        printf("The stack is empty.\n\n");
    else {
        printf("The stack is:\n");

        while (currentPtr != NULL) {
            printf("%d --> ", currentPtr->data);
            currentPtr = currentPtr->nextPtr;
        }

        printf("NULL\n\n");
    }
}

```

Fig. 12.8 Un programa de pilas simple (parte 2 de 3).


```
/* Is the stack empty? */
int isEmpty(STACKNODEPTR topPtr)
{
    return topPtr == NULL;
}
```

Fig. 12.8 Un programa de pilas simple (parte 3 de 3).

```
Enter choice:
1 to push a value on the stack
2 to pop a value off the stack
3 to end program
? 1

Enter an integer: 5
The stack is:
5 -> NULL

? 1

Enter an integer: 6
The stack is:
6 -> 5 -> NULL

? 1

Enter an integer: 4
The stack is:
4 -> 6 -> 5 -> NULL

? 2
The popped value is 4.
The stack is:
6 -> 5 -> NULL

? 2
The popped value is 6.
The stack is:
5 -> NULL

? 2
The popped value is 5.
The stack is empty.

? 2
The stack is empty.

? 4
Invalid choice.

Enter choice:
1 to push a value on the stack
2 to pop a value off the stack
3 to end program
? 3
End of run.
```

Fig. 12.9 Salida de muestra correspondiente al programa de la figura 12.8.

Las manipulaciones que involucran a `*topPtr` modifican el valor de `stackPtr` en `main`. La figura 12.10 ilustra la función `push`. La parte a) de la figura muestra la pila y el nuevo nodo, antes de la operación `push`. En la parte b) las flechas con líneas punteadas ilustran los pasos 2 y 3 de la operación `push`, que le permiten al nodo que contiene 12 convertirse en la nueva parte superior de la pila. La función `pop` retira un nodo de la parte superior de la pila. Note que `main` determina, antes de llamar a `pop`, si la pila está vacía. La operación `pop` consiste de cinco pasos.

- 1) Asigna `*topPtr` a `tempPtr` (`tempPtr` se utilizará para liberar memoria no necesaria).
- 2) Asigna `(*topPtr) -> data` a `popValue` (guarda el valor almacenado en el nodo superior).
- 3) Asigna `(*topPtr) -> nextPtr` a `*topPtr` (asigna `*topPtr` la dirección del nuevo nodo superior).
- 4) Libera la memoria a la cual apunta `tempPtr`.
- 5) Regresa `popValue` al llamador (`main`, en el programa de la figura 12.8).

La figura 12.11 ilustra el uso de la función `pop`. La parte a) muestra la pila antes de la operación anterior `push`. La parte b) muestra `tempPtr` apuntando al primer nodo de la pila y `topPtr` apuntando al segundo nodo de la misma. La función `free` es utilizada para liberar la memoria a la cual apunta `tempPtr`.

Las pilas tienen muchas aplicaciones interesantes. Por ejemplo, siempre que se hace una llamada de función, la función llamada debe saber cómo regresar a su llamador, por lo que la dirección de regreso es introducida en una pila. Si ocurre una serie de llamadas de función, los valores de regreso sucesivos son introducidos en la pila, en orden de últimas entradas, primeras salidas, de forma tal que cada función pueda regresar a su llamador. Las pilas aceptan llamadas de función recursivas, de la misma forma que llamadas normales no recursivas.

Las pilas contienen el espacio creado para variables automáticas en cada invocación de una función. Cuando la función regresa a su llamador, el espacio para las variables automáticas de dicha función es retirado (popped off) de la pila, y dichas variables dejan de ser conocidas para el programa.

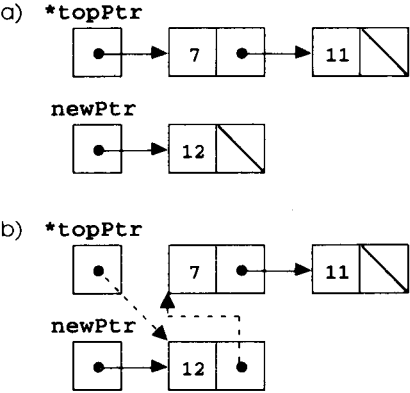
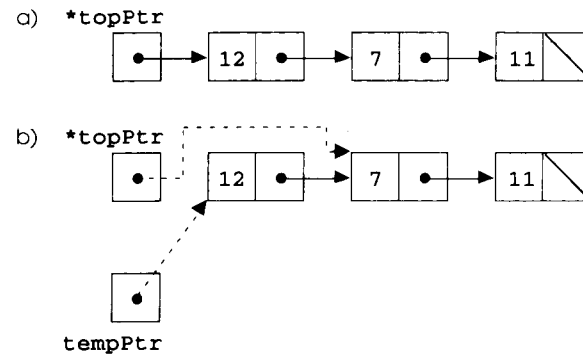


Fig. 12.10 La operación push.

Fig. 12.11 La operación **pop**.

Las pilas son utilizadas por los compiladores en el proceso de evaluar expresiones y de generar código de lenguaje máquina. En los ejercicios se exploran varias aplicaciones de las pilas.

12.6 Colas de espera

Otra estructura de datos común es la *cola de espera*. Una cola de espera es similar a una línea de pagos en un supermercado —la primera persona en la línea es atendida primero, y los otros clientes entran en la línea únicamente por la parte final y esperan para ser atendidos. Los nodos de la cola son eliminados sólo de la parte delantera o *cabeza* de la cola, y son incluidos o insertos únicamente en la *parte trasera* de la cola. Por esta razón, una cola se conoce como una estructura de datos de *primeras entradas, primeras salidas (FIFO por first-in, first-out)*. Las operaciones de insertar y de retirar se conocen como *enqueue* y *dequeue*.

Las colas tienen muchas aplicaciones en sistemas de cómputo. Muchas computadoras tienen únicamente un solo procesador, de tal forma que sólo un usuario puede ser servido a la vez. Las entradas de los demás usuarios son colocados en una cola. Cada entrada avanza en forma gradual hacia el frente de la cola, conforme los usuarios reciben servicio. La entrada que aparece en la parte delantera de la cola es la siguiente a recibir servicio.

Las colas también se utilizan para apoyar colas de impresión. Un entorno de multiusuario pudiera tener una sola impresora. Muchos usuarios podrían estar generando salidas para impresión. Si la impresora está ocupada, aún así se pueden generar otras salidas. Estas quedan en “espera” en el disco, donde esperan en una cola hasta que la impresora quede disponible.

En las redes de computadoras los paquetes de información también esperan en colas. Cada vez que un paquete llega a un nodo de red, debe ser encaminado al siguiente nodo de red, siguiendo una trayectoria hacia el destino final del paquete. El nodo de encaminamiento envía un paquete a la vez, por lo que los paquetes adicionales quedan en cola, hasta que el encaminador pueda enviarlos. En la figura. 12.12 se ilustra una cola con varios nodos. Note los apuntadores a la cabeza y a la parte trasera de la cola.

Error común de programación 12.7

No definir a `NULL` el enlace en el último nodo de una cola.

El programa de la figura 12.13 (cuya salida aparece en la figura 12.14) ejecuta manipulaciones de colas. El programa presenta varias opciones: insertar un nodo en la cola (función **enqueue**), eliminar o retirar un nodo de la cola (función **dequeue**) y terminar el programa.

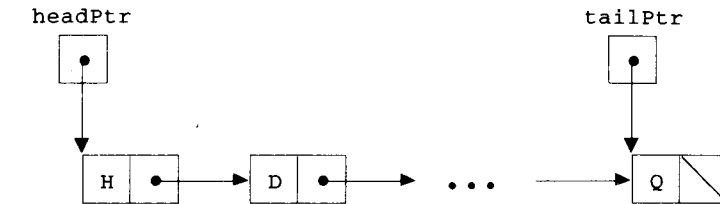


Fig. 12.12 Representación gráfica de una cola.

```
/* Operating and maintaining a queue */

#include <stdio.h>
#include <stdlib.h>

struct queueNode { /* self-referential structure */
    char data;
    struct queueNode *nextPtr;
};

typedef struct queueNode QUEUENODE;
typedef QUEUENODE *QUEUENODEPTR;

/* function prototypes */
void printQueue(QUEUENODEPTR);
int isEmpty(QUEUENODEPTR);
char dequeue(QUEUENODEPTR *, QUEUENODEPTR *);
void enqueue(QUEUENODEPTR *, QUEUENODEPTR *, char);
void instructions(void);

main()
{
    QUEUENODEPTR headPtr = NULL, tailPtr = NULL;
    int choice;
    char item;

    instructions();
    printf("? ");
    scanf("%d", &choice);

    while (choice != 3) {

        switch(choice) {

            case 1:
                printf("Enter a character: ");
                scanf("\n%c", &item);
                enqueue(&headPtr, &tailPtr, item);
                printQueue(headPtr);
                break;
```

Fig. 12.13 Procesamiento de una cola (parte 1 de 3).

```

        case 2:
            if (!isEmpty(headPtr)) {
                item = dequeue(&headPtr, &tailPtr);
                printf("%c has been dequeued.\n", item);
            }

            printQueue(headPtr);
            break;

        default:
            printf("Invalid choice.\n\n");
            instructions();
            break;
    }

    printf("? ");
    scanf("%d", &choice);
}

printf("End of run.\n");
return 0;
}

void instructions(void)
{
    printf("Enter your choice:"
        " 1 to add an item to the queue"
        " 2 to remove an item from the queue"
        " 3 to end");
}

void enqueue(QUEUENODEPTR *headPtr, QUEUENODEPTR *tailPtr,
             char value)
{
    QUEUENODEPTR newPtr;

    newPtr = malloc(sizeof(QUEUENODE));

    if (newPtr != NULL) {
        newPtr->data = value;
        newPtr->nextPtr = NULL;

        if (isEmpty(*headPtr))
            *headPtr = newPtr;
        else
            (*tailPtr)->nextPtr = newPtr;

        *tailPtr = newPtr;
    }
    else
        printf("%c not inserted. No memory available.\n", value);
}

```

Fig. 12.13 Procesamiento de una cola (parte 2 de 3).

```

char dequeue(QUEUENODEPTR *headPtr, QUEUENODEPTR *tailPtr)
{
    char value;
    QUEUENODEPTR tempPtr;

    value = (*headPtr)->data;
    tempPtr = *headPtr;
    *headPtr = (*headPtr)->nextPtr;

    if (*headPtr == NULL)
        *tailPtr = NULL;

    free(tempPtr);
    return value;
}

int isEmpty(QUEUENODEPTR headPtr)
{
    return headPtr == NULL;
}

void printQueue(QUEUENODEPTR currentPtr)
{
    if (currentPtr == NULL)
        printf("Queue is empty.\n\n");
    else {
        printf("The queue is:\n");

        while (currentPtr != NULL) {
            printf("%c --> ", currentPtr->data);
            currentPtr = currentPtr->nextPtr;
        }

        printf("NULL\n\n");
    }
}

```

Fig. 12.13 Procesamiento de una cola (parte 3 de 3).

La función **enqueue** recibe desde **main** tres argumentos: la dirección del apuntador a la cabeza de la cola, la dirección del apuntador a la parte trasera de la cola, y el valor a ser inserto en la cola. La función está formada de tres pasos:

- 1) Para crear un nuevo nodo: llama **malloc**, asigna a **newPtr** la posición asignada de memoria, asigna el valor que se va a insertar en la cola a **newPtr->data** y asigna **NULL** a **newPtr->nextPtr**.
- 2) Si la cola está vacía, asigna **newPtr** a ***headPtr**; de no ser así, asigna el apuntador **newPtr** a **(*tailPtr)->nextPtr**.
- 3) Asigna **newPtr** a ***tailPtr**.

La figura 12.15 ilustra una operación **enqueue**. La parte a) de la figura muestra la cola y el nuevo nodo antes de la operación. Las flechas con líneas punteadas de la parte b) ilustran los pasos 2 y 3 de la función **enqueue** que permiten añadir un nuevo nodo al final de una cola que no esté vacía.

```
Enter your choice:
  1 to add an item to the queue
  2 to remove an item from the queue
  3 to end
? 1
Enter a character: A
The queue is:
A -> NULL

? 1
Enter a character: B
The queue is:
A -> B -> NULL

? 1
Enter a character: C
The queue is:
A -> B -> C -> NULL

? 2
A has been dequeued.
The queue is:
B -> C -> NULL

? 2
B has been dequeued.
The queue is:
C -> NULL

? 2
C has been dequeued.
Queue is empty.

? 2
Queue is empty.

? 4
Invalid choice.

Enter your choice:
  1 to add an item to the queue
  2 to remove an item from the queue
  3 to end
? 3
End of run.
```

Fig. 12.14 Salida de muestra del programa 12.13.

La función **dequeue** recibe como argumentos la dirección del apuntador a la cabeza de la cola y la dirección del apuntador a la parte trasera de la cola, y retira el primer nodo de la cola. La operación **dequeue** consiste de seis pasos:

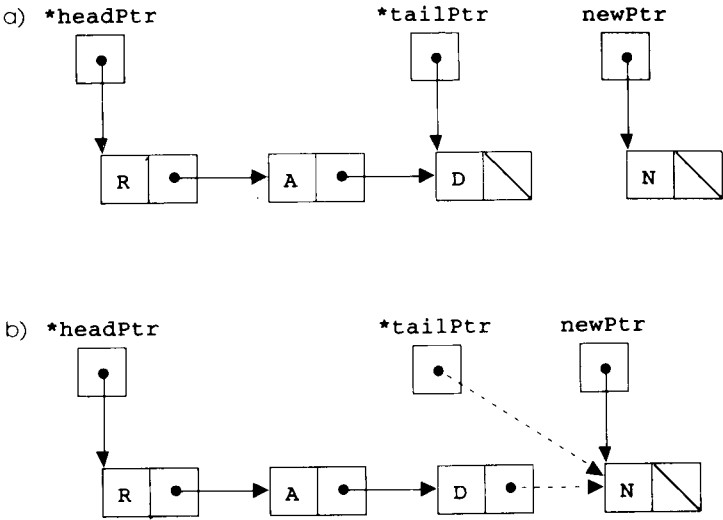


Fig. 12.15 Representación gráfica de la operación **enqueue**.

- 1) Asigna **(*headPtr) ->data** a **value** (guarda los datos)
- 2) Asigna ***headPtr** a **tempPtr** (**tempPtr** es utilizado para liberar, mediante **free**, la memoria no necesaria).
- 3) Asigna **(*headPtr) ->nextPtr** a ***headPtr** (***headPtr** apunta ahora al primer nodo en la cola).
- 4) Si ***headPtr** es **NULL**, asigna **NULL** a ***tailPtr**
- 5) Libera la memoria a la cual apunta **tempPtr**.
- 6) Regresa **value** al llamador (la función de **dequeue** es llamada desde **main** en el programa de la figura 12.13).

La figura 12.16 ilustra la función **dequeue**. La parte a) muestra la cola antes de la operación **enqueue** anterior. La parte b) muestra **tempPtr** apuntando al nodo a retirarse (**dequeue**), y **headPtr** apuntando al nuevo primer nodo de la cola. La función **free** es utilizada para recuperar la memoria a la cual **tempPtr** apunta.

12.7 Árboles

Las listas enlazadas, las pilas y las colas son *estructuras lineales de datos*. Un árbol es una estructura no lineal y de dos dimensiones de datos, con propiedades especiales. Los nodos de los árboles contienen dos o más enlaces. Esta sección analiza los *árboles binarios* (figura 12.17) —árboles cuyos nodos todos ellos contienen dos enlaces (ninguno, uno, o ambos de los cuales pudieran ser **NULL**). El *nodo raíz* es el primer nodo de un árbol. Cada enlace en el nodo raíz se refiere a un *hijo*. El *hijo izquierdo* es el primer nodo en el *subárbol izquierdo* y el *hijo derecho* es el primer nodo en el *subárbol derecho*. Los hijos de un nodo se conocen como *descendientes*. Un nodo sin hijos se conoce como *nodo de hoja*. Los científicos de la computación normalmente dibujan los árboles partiendo del nodo raíz hacia abajo —en forma exactamente opuesta a los árboles en la naturaleza.

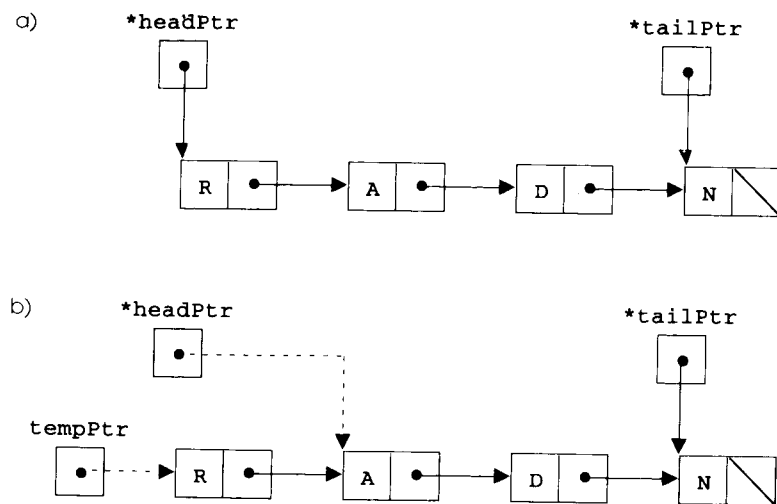


Fig. 12.16 Representación gráfica de la operación dequeue.

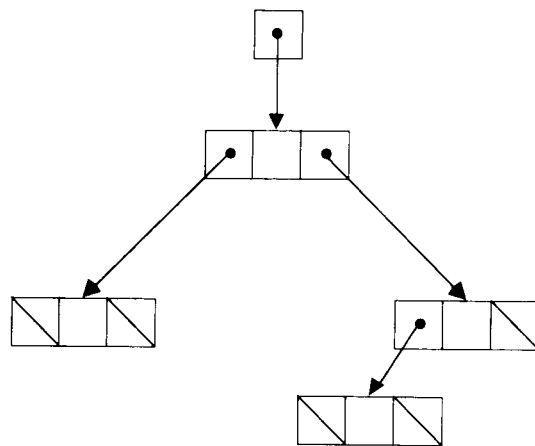


Fig. 12.17 Representación gráfica de un árbol binario.

En esta sección, se creará un árbol binario especial conocido como un *árbol de búsqueda binario*. Un árbol de búsqueda binario (que no tiene valores duplicados de nodos) tienen la característica que los valores en cualquier subárbol izquierdo son menores que el valor en sus nodos padre, y los valores en cualquier subárbol derecho son mayores que el valor en sus nodos padre. En la figura 12.18 se ilustra un árbol de búsqueda binario con 12 valores. Note que la forma del árbol de búsqueda binario que corresponde a un conjunto de datos puede variar, dependiendo del orden en el cual los valores están insertos dentro del árbol.

Error común de programación 12.8

No definir a **NULL** los enlaces en los nodos de hoja de un árbol.

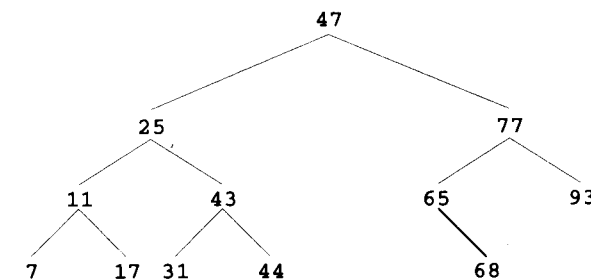


Fig. 12.18 Un árbol de búsqueda binario.

El programa de la figura 12.19 (cuya salida se muestra en la figura 12.20) crea un árbol de búsqueda binario y lo recorre de tres formas —*enorden*, *preorden* y *postorden*. El programa genera 10 números aleatorios e inserta cada uno de ellos en el árbol, a excepción de los valores duplicados, que son descartados.

Las funciones utilizadas en la figura 12.19, para crear un árbol de búsqueda binario y recorrer el árbol, son recursivas. La función **insertNode** recibe como argumentos la dirección del árbol y un entero para almacenarse en el árbol. En un árbol de búsqueda binario un nodo puede ser únicamente inserto como nodo de hoja. Los pasos para insertar un nodo en un árbol de búsqueda binario, son como sigue:

- 1) Si ***treePtr** es **NULL**, crea un nuevo nodo. Llame **malloc**, asigna la memoria asignada a ***treePtr**, asigna a **(*treePtr) -> data** el entero a almacenarse, asigna a **(*treePtr) -> leftPtr** y **(*treePtr) -> rightPtr** el valor **NULL** y devuelve o regresa el control al llamador (ya sea a **main** o a una llamada anterior a **insertNode**).
- 2) Si el valor de ***treePtr** no es **NULL**, y el valor a insertarse es menor que **(*treePtr) -> data**, se llama a la función **insertNode** con la dirección de **(*treePtr) -> leftPtr**. De no ser así, se llama a la función **insertNode** con la dirección de **(*treePtr) -> rightPtr**. Se continúan los pasos recursivos hasta que se encuentre un apuntador **NULL**, entonces se ejecutará el paso 1) para insertar el nuevo nodo.

Las funciones **inOrder**, **preOrder** y **postOrder** cada una de ellas recibe un árbol (es decir, el apuntador al nodo raíz del árbol) y recorren el árbol.

Los pasos para un recorrido **inOrder** son:

- 1) Recorrer el subárbol izquierdo **inOrder**.
- 2) Procesar el valor en el nodo.
- 3) Recorrer el subárbol derecho **inOrder**.

El valor en un nodo no es procesado en tanto no sean procesados los valores de su subárbol izquierdo. El recorrido **inOrder** del árbol en la figura 12.21 es

6 13 17 27 33 42 48

Note que el recorrido **inOrder** de un árbol de búsqueda binario imprime los valores de nodo en valor ascendente. El proceso de crear un árbol de búsqueda binario, de hecho ordena los datos —y por lo tanto este proceso se llama la *clasificación de árbol binario*.

```

/* Create a binary tree and traverse it
preorder, inorder, and postorder */

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct treeNode {
    struct treeNode *leftPtr;
    int data;
    struct treeNode *rightPtr;
};

typedef struct treeNode TREENODE;
typedef TREENODE *TREENODEPTR;

void insertNode(TREENODEPTR *, int);
void inOrder(TREENODEPTR);
void preOrder(TREENODEPTR);
void postOrder(TREENODEPTR);

main()
{
    int i, item;
    TREENODEPTR rootPtr = NULL;

    srand(time(NULL));

    /* attempt to insert 10 random values between 0 and 14 in the tree */
    printf("The numbers being placed in the tree are:\n");

    for (i = 1; i <= 10; i++) {
        item = rand() % 15;
        printf("%3d", item);
        insertNode(&rootPtr, item);
    }

    /* traverse the tree preOrder */
    printf("\n\nThe preOrder traversal is:\n");
    preOrder(rootPtr);

    /* traverse the tree inOrder */
    printf("\n\nThe inOrder traversal is:\n");
    inOrder(rootPtr);

    /* traverse the tree postOrder */
    printf("\n\nThe postOrder traversal is:\n");
    postOrder(rootPtr);

    return 0;
}

```

Fig. 12.19 Cómo crear y recorrer un árbol binario (parte 1 de 2).

```

void insertNode(TREENODEPTR *treePtr, int value)
{
    if (*treePtr == NULL) { /* *treePtr is NULL */
        *treePtr = malloc(sizeof(TREENODE));

        if (*treePtr != NULL) {
            (*treePtr)->data = value;
            (*treePtr)->leftPtr = NULL;
            (*treePtr)->rightPtr = NULL;
        }
        else
            printf("%d not inserted. No memory available.\n",
                value);
    }
    else
        if (value < (*treePtr)->data)
            insertNode(&((*treePtr)->leftPtr), value);
        else
            if (value > (*treePtr)->data)
                insertNode(&((*treePtr)->rightPtr), value);
            else
                printf("dup");
}

void inOrder(TREENODEPTR treePtr)
{
    if (treePtr != NULL) {
        inOrder(treePtr->leftPtr);
        printf("%3d", treePtr->data);
        inOrder(treePtr->rightPtr);
    }
}

void preOrder(TREENODEPTR treePtr)
{
    if (treePtr != NULL) {
        printf("%3d", treePtr->data);
        preOrder(treePtr->leftPtr);
        preOrder(treePtr->rightPtr);
    }
}

void postOrder(TREENODEPTR treePtr)
{
    if (treePtr != NULL) {
        postOrder(treePtr->leftPtr);
        postOrder(treePtr->rightPtr);
        printf("%3d", treePtr->data);
    }
}

```

Fig. 12.19 Cómo crear y recorrer un árbol binario (parte 2 de 2).

```
The numbers being placed in the tree are:
7 8 0 6 14 1 0dup 13 0dup 7dup

The preOrder traversal is:
7 0 6 1 8 14 13

The inOrder traversal is:
0 1 6 7 8 13 14

The postOrder traversal is:
1 6 0 13 14 8 7
```

Fig. 12.20 Salida de muestra correspondiente al programa de la figura 12.19.

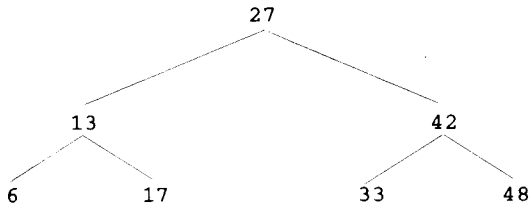


Fig. 12.21 Un árbol de búsqueda binario.

Los pasos para un recorrido **preOrder** son:

- 1) Procesar el valor en el nodo.
- 2) Recorrer el subárbol izquierdo **preOrder**.
- 3) Recorrer el subárbol derecho **preOrder**.

El valor en cada nodo es procesado conforme se pasa por cada nodo. Después de que se procese el valor en un nodo dado, son procesados los valores del subárbol izquierdo, y a continuación los valores en el subárbol derecho. El recorrido **preOrder** del árbol en la figura 12.21 es:

27 13 6 17 42 33 48

Los pasos para un recorrido **postOrder** son:

- 1) Recorrer el subárbol izquierdo **postOrder**.
- 2) Recorrer el subárbol derecho **postOrder**.
- 3) Procesar el valor en el nodo.

El valor en cada nodo no se imprime hasta que sean impresos los valores de sus hijos. El recorrido **postOrder** del árbol de la figura 12.21 es:

6 17 13 33 48 42 27

El árbol de búsqueda binario facilita la eliminación de duplicados. Conforme se crea el árbol, cualquier intento para insertar un valor duplicado será detectado porque en cada una de las comparaciones un duplicado seguirá las mismas decisiones “ir a la izquierda” o “ir a la derecha” que utilizó el valor original. Entonces, el duplicado eventualmente será comparado con un nodo que contenga el mismo valor. Llegado a este punto el valor duplicado pudiera simplemente ser descartado.

También es rápido buscar en un árbol binario un valor que coincida con un valor clave. Si el árbol es denso, entonces cada nivel contendrá aproximadamente dos veces tantos elementos como el nivel anterior. Por lo tanto un árbol de búsqueda binario con n elementos tendría un máximo de $\log_2 n$ (logaritmo de base 2 de n niveles) y, por lo tanto, tendrían que efectuarse un máximo $\log_2 n$ de comparaciones, ya sea para encontrar una coincidencia, o para determinar que no existe ninguna. Esto significa, por ejemplo, que al buscar en un árbol de búsqueda binario de 1000 elementos (densamente empacados), no es necesario llevar a cabo más de 10 comparaciones, porque $2^{10} > 1000$. Al buscar en un árbol de búsqueda binario de 1,000,000 (densamente empacados), no es necesario llevar a cabo más de 20 comparaciones, porque $2^{20} > 1,000,000$.

En los ejercicios se presentan algoritmos para otras operaciones de árboles de búsqueda binarios, como es borrar un elemento de un árbol binario, imprimir un árbol binario en un formato de árbol de dos dimensiones, y ejecutar un recorrido en orden de niveles de un árbol binario. El recorrido de orden de niveles de un árbol binario pasa por los nodos de un árbol, renglón por renglón, empezando por el nivel del nodo raíz. En cada uno de los niveles de árbol, se pasa por los nodos de izquierda a derecha. Otros ejercicios de árboles binarios incluyen permitir que un árbol de búsqueda binario contenga valores duplicados, insertar valores de cadenas en un árbol binario, y determinar cuántos niveles están incluidos en un árbol binario.

Resumen

- Las estructuras autorreferenciadas contienen miembros conocidos como enlaces, que apuntan a estructuras del mismo tipo de estructura.
- Las estructuras autorreferenciadas permiten que se enlacen muchas estructuras juntas en pilas, colas, listas y árboles.
- La asignación dinámica de memoria reserva un bloque de bytes en la memoria para almacenar un objeto de datos durante la ejecución del programa.
- La función **malloc** toma como argumento un número de bytes a asignarse, y regresa un apuntador **void** a la memoria asignada. La función **malloc** se utiliza normalmente junto con el operador **sizeof**. El operador **sizeof** determina el tamaño en bytes de la estructura para la cual se está asignando memoria.
- La función **free** cancela la asignación de memoria.
- Una lista enlazada es una colección de datos almacenados en un grupo de estructuras autorreferenciadas conectadas.
- Una lista enlazada es una estructura dinámica de datos —la longitud de la lista puede aumentarse o reducirse conforme sea necesario.
- Las listas enlazadas pueden continuar creciendo en tanto exista memoria disponible.
- Las listas enlazadas proporcionan un mecanismo para insertar y borrar datos en forma simple mediante la reasignación de apuntadores.

- Las pilas y las colas son versiones especializadas de una lista enlazada.
- Se añaden nuevos nodos a una pila y son retirados nodos de una pila únicamente de su parte superior. Por esta razón, se conoce una pila como una estructura de datos de últimas entradas, primeras salidas (LIFO).
- El miembro de enlace en el último nodo de la pila se define a **NULL** para indicar la parte inferior de la pila.
- Las dos operaciones primarias utilizadas para manipular una pila son **push** y **pop**. La operación **push** crea un nuevo nodo y lo coloca en parte superior de la pila. La operación **pop** retira un nodo de la parte superior de la pila, libera la memoria que estaba asignada al nodo retirado, y regresa el valor retirado.
- En una estructura de datos de cola, los nodos son retirados de la cabeza y añadidos a la parte trasera. Por esta razón, una cola se conoce como una estructura de datos de primeras entradas, primeras salidas (FIFO). Las operaciones de añadir y de retirar se conocen como **enqueue** y **dequeue**.
- Los árboles son estructuras de datos más complejas que las listas enlazadas, las colas y las pilas. Los árboles son estructuras de datos de dos dimensiones, que requieren de dos o más enlaces por nodo.
- Los árboles binarios contienen dos enlaces por nodo.
- El nodo raíz es el primer nodo en el árbol.
- Cada uno de los apuntadores en el nodo raíz se refiere a un hijo. El hijo izquierdo es el primer nodo en el subárbol izquierdo, y el hijo derecho es el primer nodo en el subárbol derecho. Los hijos de un nodo se conocen como descendientes. Si un nodo no tiene ningún descendiente, se le conoce como un nodo de hoja.
- Un árbol de búsqueda binario tiene la característica que tiene el valor en el hijo izquierdo del nodo menor que el valor del nodo padre, y el valor en el hijo derecho de un nodo es mayor o igual que el valor del nodo padre. Si se puede determinar que no existen valores de datos duplicados, simplemente el valor en el hijo derecho es mayor que el valor en el nodo padre.
- Un recorrido enorden de un árbol binario recorre enorden el subárbol izquierdo, procesa el valor en el nodo, y recorre enorden el subárbol derecho. El valor en un nodo no será procesado hasta que hayan sido procesados los valores en su subárbol izquierdo.
- Un recorrido en preorden procesa el valor en el nodo, recorre en preorden el subárbol izquierdo, y recorre en preorden el subárbol derecho. El valor en cada nodo se procesa conforme se encuentra cada nodo.
- Un recorrido en postorden recorre en postorden el subárbol izquierdo, recorre en postorden el subárbol derecho, y procesa el valor en el nodo. El valor en cada nodo no es procesado hasta que sean procesados los valores en ambos de sus subárboles.

Terminología

árbol de búsqueda binaria	borrar un nodo
árbol binario	dequeue
clasificación de árbol binario	doble indirección
nodo hijo	estructuras dinámicas de datos
descendientes	asignación dinámica de memoria

enqueue	recorrido postorden
FIFO (primeras entradas, primeras salidas)	función predicada
free	recorrido preorden
cabeza de una cola	push
recorrido enorden	cola
inserción de un nodo	hijo derecho
nodo de hoja	subárbol derecho
hijo izquierdo	nodo raíz
subárbol izquierdo	estructura autorreferenciada
LIFO (últimas entradas, primeras salidas)	descendencias
estructura lineal de datos	sizeof
lista enlazada	pila
malloc (asignar memoria)	subárbol
nodo	parte trasera de una cola
estructura de datos no lineal	parte superior
apuntador NULL	recorrido
nodo padre	árbol
apuntador a un apuntador	pasar por un nodo
pop	

Errores comunes de programación

- 12.1 No establecer a **NULL** el enlace en el último nodo de una lista.
- 12.2 Suponer que el tamaño de una estructura es simplemente la suma de los tamaños de sus miembros.
- 12.3 No regresar memoria dinámicamente asignada cuando ésta ya no es necesaria, puede hacer que el sistema se quede sin memoria prematuramente. Esto se conoce a veces como “fuga de memoria”.
- 12.4 Utilizando **malloc**, liberar memoria no dinámicamente asignada.
- 12.5 Referirse a memoria que ya ha sido liberada.
- 12.6 No definir a **NULL** el enlace en el nodo inferior de una pila.
- 12.7 No definir a **NULL** el enlace en el último nodo de una cola.
- 12.8 No definir a **NULL** los enlaces en los nodos de hoja de un árbol.

Prácticas sanas de programación

- 12.1 Para determinar el tamaño de una estructura utilice el operador **sizeof**.
- 12.2 Al utilizar **malloc**, compruebe si es **NULL** el valor de regreso de apuntador. Si la memoria solicitada no ha sido asignada imprima un mensaje de error.
- 12.3 Cuando ya no se requiera memoria que fue dinámicamente asignada, utilice **free**, para regresar esta memoria inmediatamente al sistema.
- 12.4 Asigne **NULL** al miembro de enlace de un nuevo nodo. Los apuntadores deben ser inicializados antes de ser utilizados.

Sugerencias de rendimiento

- 12.1 Podría declararse un arreglo que contenga más elementos que el número esperado de elementos de datos, pero esto puede desperdiciar memoria. En estas situaciones las listas enlazadas pueden obtener una mejor utilización de la memoria.
- 12.2 Puede resultar muy tardado insertar y eliminar en un arreglo ya ordenado —deberán ser desplazados en forma apropiada. Todos los elementos que sigan al elemento insertado o borrado.

- 12.3 Los elementos de un arreglo se almacenan en forma contigua en memoria. Esto permite acceso inmediato a cualquier arreglo del elemento, porque la dirección de cualquier elemento puede ser calculada directamente, basada en su posición en relación con el principio del arreglo. Las listas enlazadas no proporcionan un acceso inmediato como éste a sus elementos.
- 12.4 Tratándose de estructuras de datos que crecen o se reducen en tiempo de ejecución, es posible ahorrar memoria utilizando asignación dinámica de memoria (en vez de los arreglos). Recuerde, sin embargo, que los apuntadores toman espacio, y que la asignación dinámica de memoria incurre en sobrecarga por las llamadas de función.

Sugerencia de portabilidad

- 12.1 El tamaño de una estructura no es necesariamente la suma de los tamaños de sus miembros. Esto es debido a varios requisitos de alineación de límites, que son dependientes de la máquina (vea el capítulo 10).

Ejercicios de autoevaluación

- 12.1 Llene cada uno de los siguientes espacios vacíos:
- a) Una estructura auto _____ se utiliza para formar estructuras dinámicas de datos.
 - b) La función _____ se utiliza para asignar dinámicamente la memoria.
 - c) Una _____ es una versión especializada de una lista enlazada, en la cual los nodos pueden ser insertos y borrados únicamente a partir del principio de la lista.
 - d) Las funciones que no modifican una lista enlazada, pero que simplemente analizan la lista se conocen como _____.
 - e) Una cola de espera se conoce como una estructura de datos _____ porque los primeros nodos insertos son los primeros nodos retirados.
 - f) El apuntador al siguiente nodo en una lista enlazada se conoce como un _____.
 - g) La función _____ se utiliza para recuperar memoria dinámicamente asignada.
 - h) Una _____ es una versión especializada de una lista enlazada en la cual los nodos pueden ser insertos únicamente al principio de la lista y retirados o borrados únicamente de la parte final de la lista.
 - i) Una _____ es una estructura de datos de dos dimensiones no lineal, que contiene nodos con dos o más enlaces.
 - j) Una pila se conoce como una estructura de datos _____ porque el último nodo insertado es el primer nodo retirado.
 - k) Los nodos de un árbol _____ contienen dos miembros enlazados.
 - l) El primer nodo de un árbol es el nodo _____.
 - m) Cada enlace en un nodo de árbol apunta a un _____ o a un _____ de dicho nodo.
 - n) Un nodo de árbol que no tiene hijos se conoce como un nodo _____.
 - o) Los tres algoritmos de recorrido para un árbol binario son _____, _____ y _____.

12.2 ¿Cuáles son las diferencias entre una lista enlazada y una pila?

12.3 ¿Cuáles son las diferencias entre una pila y una cola de espera?

12.4 Escriba un enunciado, o un conjunto de enunciados para llevar a cabo cada uno de los siguientes. Suponga que todas las manipulaciones ocurren en `main` (por lo tanto, no son necesarias direcciones a variables de apuntador) y suponga las definiciones siguientes:

```
struct gradeNode {
    char lastName[20];
    float grade;
    struct gradeNode *nextPtr;
};
```

```
typedef struct gradeNode GRADENODE;
typedef GRADENODE *GRADENODEPTR;
```

- a) Crea un apuntador al principio de la lista llamado `startPtr`. La lista está vacía.
- b) Crea un nuevo nodo del tipo `GRADENODE` al cual apunta el apuntador `newPtr` del tipo `GRADENODEPTR`. Asigne la cadena "Jones" al miembro `lastName` y el valor 91.5 al miembro `grade` (utilice `strcpy`). Incluya todas las declaraciones y enunciados necesarios.
- c) Suponga que la lista a la cual apunta `startPtr` está formada actualmente de dos nodos —uno que contiene "Jones" y uno que contiene "Smith". Los nodos están en orden alfabético. Proporcione los enunciados necesarios para insertar otros nodos que contengan los siguientes datos para `lastName` y `grade`:

"Adams"	85.0
"Thompson"	73.5
"Pritchard"	66.5

Utilice los apuntadores `previousPtr`, `currentPtr` y `newPtr` para llevar a cabo las inserciones. Declare hacia donde apuntan `previousPtr`, y `currentPtr` antes de cada inserción. Suponga que `newPtr` siempre apunta al nuevo nodo, y que el nuevo nodo ya ha sido asignado con los datos.

- d) Escriba un ciclo `while` que imprima los datos de cada nodo de la lista. Utilice el apuntador `currentPtr` para pasar a lo largo de la misma.
- e) Escriba un ciclo `while` que borre todos los nodos en la lista y que libere la memoria asociada con cada uno de los nodos. Utilice el apuntador `currentPtr` y el apuntador `tempPtr` para caminar a lo largo de la lista y liberar memoria respectivamente.

12.5 Proporcione manualmente los recorridos enorden, preorden y postorden del árbol de búsqueda binario de la Figura 12.22.

Respuestas a los ejercicios de autoevaluación

12.1 a) referenciados. b) `malloc`. c) pila. d) predicados. e) FIFO. f) enlace. g) `free`. h) cola de espera. i) árbol. j) LIFO. k) binario. l) raíz. m) hijo o subárbol. n) hoja. o) enorden, preorden y postorden.

12.2 Es posible insertar un nodo en cualquier parte de una lista enlazada, y eliminar un nodo de cualquier parte de una lista enlazada. Sin embargo, los nodos en una pila pueden únicamente ser insertos en la parte superior de la misma y eliminados también de la parte superior de la misma.

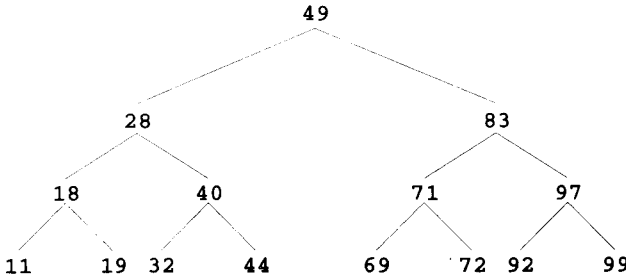


Fig. 12.22 Un árbol de búsqueda binario de 15 nodos.

12.3 Una cola tiene apuntadores tanto a su cabeza como a su parte trasera, de tal forma que los nodos pueden ser insertos en la parte trasera y borrados de la cabeza. Una pila tiene un solo apuntador a la parte superior de la misma donde se ejecuta tanto la inserción como el borrado de los nodos.

12.4

```
a) GRADENODEPTR startPtr = NULL;
b) GRADENODEPTR newPtr;
   newPtr = malloc(sizeof(GRADENODE));
   strcpy(newPtr->lastName, "Jones");
   newPtr->grade = 91.5;
   newPtr->nextPtr = NULL
c) Para insertar "Adams";
   previousPtr es NULL, current Ptr apunta al primer elemento en la lista.
   newPtr->nextPtr = currentPtr;
   startPtr = newPtr;

   Para insertar "Thompson":
   previousPtr apunta al último elemento en la lista (que contiene "Smith")
   current Ptr es NULL
   newPtr->nextPtr = currentPtr;
   previousPtr->nextPtr = newPtr;

   Para insertar "Pritchard"
   previousPtr apunta al nodo que contiene "Jones"
   currentPtr apunta al nodo que contiene "Smith".
   newPtr->nextPtr = currentPtr;
   previousPtr->nextPtr = newPtr;
d) current Ptr = startPtr;
   while (current Ptr != NULL) {
       printf("Lastname = %s\nGrade = %6.2f\n",
           currentPtr->lastName, currentPtr->grade);
       currentPtr = currentPtr->nextPtr;
   }
e) currentPtr = startPtr;
   while (current Ptr != NULL) {
       tempPtr = currentPtr;
       currentPtr = currentPtr->nextPtr;
       free(tempPtr);
   }
   startPtr = NULL;
```

12.5 El recorrido enorden es:

11 18 19 28 32 40 44 49 69 71 72 83 92 97 99

El recorrido preorden es:

49 28 18 11 19 40 32 44 83 71 69 72 97 92 99

El recorrido postorden es:

11 19 18 32 44 40 28 69 72 71 92 99 97 83 49

Ejercicios

12.6 Escriba un programa que concatene dos listas enlazadas de caracteres. El programa deberá incluir la función **concatenate** que toma como argumentos apuntadores a ambas listas y que concatena la segunda lista a la primera.

12.7 Escriba un programa que combine dos listas ordenadas de enteros en una sola lista ordenada de enteros. La función **merge** deberá recibir apuntadores al primer nodo de cada una de las listas a combinarse, y deberá regresar un apuntador al primer nodo de la lista combinada.

12.8 Escriba un programa que inserte 25 enteros al azar desde 0 hasta 100 en orden en una lista enlazada. El programa deberá calcular la suma de los elementos, y el promedio en punto flotante de los mismos.

12.9 Escriba un programa que origine una lista enlazada de 10 caracteres, y a continuación origine una copia de la lista de orden inverso.

12.10 Escriba un programa que introduzca una línea de texto y utilice una pila para imprimir la línea invertida.

12.11 Escriba un programa que utilice una pila para determinar si una cadena es un palíndromo (es decir, si la cadena se deletrea en forma idéntica hacia adelante y hacia atrás). El programa deberá ignorar espacios y puntuaciones.

12.12 Las pilas son utilizadas por los compiladores para auxiliarse en el proceso de evaluar expresiones y para generar código en lenguaje máquina. En este ejercicio y en el siguiente, investigamos cómo los compiladores evalúan expresiones aritméticas formadas únicamente por constantes, operadores y paréntesis.

Los seres humanos normalmente escriben las expresiones como $3 + 4$ y $7 / 9$, en el cual el operador (aquí $+$ o $/$) se escribe entre sus operandos —esto se conoce como *notación infija*. Las computadoras "prefieren" la *notación postfija*, en la cual el operador se escribe a la derecha de sus dos operandos. Las expresiones infijas anteriores aparecerían en notación postfija como $3\ 4\ +$ y $7\ 9\ /$, respectivamente.

Para evaluar una expresión infija compleja, un compilador primero convertiría la expresión a notación postfija, y a continuación evaluaría la versión postfija de la expresión. Cada uno de estos algoritmos requiere únicamente de una pasada de izquierda a derecha en la expresión. Cada algoritmo utiliza una pila para apoyarse en esta operación, y en cada uno la pila se utiliza para fines distintos.

En este ejercicio, usted escribirá una versión en C del algoritmo de conversión de infijos a postfijos. En el siguiente ejercicio, escribirá una versión en C del algoritmo de evaluación de la expresión postfija.

Escriba un programa que convierta una expresión infija aritmética ordinaria (suponga que se ha introducido una expresión válida) con enteros de un solo dígito, como

$$(6 + 2) * 5 - 8 / 4$$

a una expresión postfija. La expresión postfija de la expresión infija anterior es

$$6\ 2\ +\ 5\ *\ 8\ 4\ /\ -$$

El programa debería leer la expresión al arreglo de caracteres **infix**, y utilizar las versiones modificadas de las funciones de pila puestas en práctica en este capítulo para crear la expresión postfija en el arreglo de caracteres **postfix**. El algoritmo para crear una expresión postfija es como sigue:

- 1) Inserte (push) un paréntesis izquierdo '(' en la parte superior de la pila.
- 2) Agregue un paréntesis derecho ')' al final de infix.
- 3) En tanto la pila no esté vacía, lea infix de izquierda a derecha y haga lo siguiente:
 Si el carácter actual en **infix** es un dígito, cópielo al siguiente elemento de **postfix**.
 Si el carácter actual en **infix** es un paréntesis izquierdo, insértelo (push) sobre la pila.
 Si el carácter actual en **infix** es un operador,
 Retire (pop) los operadores (si es que hay alguno) en la parte superior de la pila en tanto tengan precedencia igual o mayor que el operador actual, e inserte los operadores retirados en **postfix**.
 Inserte (push) el carácter actual en **infix** sobre la pila.

Si el carácter actual en **infix** es un paréntesis derecho

Retire (pop) los operadores de la parte superior de la pila e insértelos en **postfix** hasta que en la parte superior de la pila quede un paréntesis izquierdo.

Retire (pop) y descarte el paréntesis izquierdo de la pila.

Las siguientes operaciones aritméticas se permiten en una expresión:

+	adición
-	substracción
*	multiplicación
/	división
^	exponenciación
%	módulo

La pila deberá ser mantenida utilizando las siguientes declaraciones:

```
struct stackNode {
    char data;
    struct stackNode *nextPtr;
};
typedef struct stackNode STACKNODE;
typedef STACKNODE *STACKNODEPTR;
```

El programa deberá estar formado por **main** y 8 otras funciones, con los siguientes encabezados de función:

```
void convertToPostfix(char infix[], char postfix[])
    Convierta la expresión infija a notación postfija.
int isOperator(char c)
    Determinar si c es un operador.
int precedence(char operator1, char operator2)
    Determinar si la precedencia de operador1 es menor que, igual a, o mayor que la precedencia de operador2. La función regresa -1, 0 y 1, respectivamente.
void push(STACKNODEPTR *topPtr, char value)
    Insertar (push) un valor en la pila.
char pop(STACKNODEPTR *topPtr)
    Retirar (pop) un valor de la pila.
char stackTop(STACKNODEPTR topPtr)
    Regresar el valor superior de la pila sin retirar nada de la pila.
int isEmpty(STACKNODEPTR topPtr)
    Determinar si la pila está vacía.
void printStack(STACKNODEPTR topPtr)
    Imprimir la pila.
```

12.13 Escriba un programa que evalúe una expresión postfija (suponga que es válida) como

6 2 + 5 * 8 4 / -

El programa deberá leer una expresión postfija formada de dígitos y de operadores a un arreglo de caracteres. Utilizando versiones modificadas de las funciones de pila puestas anteriormente en práctica en este capítulo, el programa deberá rastrear la expresión y evaluarla. El algoritmo es como sigue:

- 1) Agregue el carácter **NULL** ('\\0') al final de la expresión postfija. Cuando se encuentre el carácter **NULL**, ya no es necesario procesamiento adicional.
- 2) En tanto no se encuentre '\\0', lea la expresión de izquierda a derecha.
Si el carácter actual es un dígito,
Inserte (push) su valor entero en la pila (el valor entero de un carácter de dígito es su valor en el conjunto de caracteres de la computadora, menos el valor de '0' en el conjunto de caracteres de la computadora).

De lo contrario, si el carácter actual es un operador,

Retire (pop) los dos elementos superiores de la pila a las variables **x** e **y**.

Calcule y **operator x**

Inserte (push) el resultado del cálculo en la pila.

- 3) Cuando en la expresión se encuentre el carácter **NULL**, retire (pop) el valor superior de la pila. Ese es el resultado de la expresión postfija.

Nota: en 2) arriba, si el operador es '/', la parte superior de la pila es 2, y el siguiente elemento en la pila es 8, entonces retira (pop) 2 a **x**, retira (pop) 8 a **y**, evalúa $8/2$ e inserta (push) el resultado con 4, de regreso a la pila. Esta nota también se aplica al operador '-'. Las operaciones aritméticas permitidas en una expresión son:

+	adición
-	substracción
*	multiplicación
/	división
^	exponenciación
%	módulo

La pila deberá ser mantenida utilizando las declaraciones siguientes:

```
struct stackNode {
    int data;
    struct stackNode *nextPtr;
};
typedef struct stackNode STACKNODE;
typedef STACKNODE *STACKNODEPTR;
```

El programa deberá consistir de **main** y de 8 otras funciones, con los siguientes encabezados de función:

```
int evaluatePostfixExpression(char *expr)
    Evalúa la expresión postfija.
int calculate(int op1, int op2, char operator)
    Evalúa la expresión op1 operator op2.
void push(STACKNODEPTR *topPtr, int value)
    Insertar (push) un valor en la pila.
int pop(STACKNODEPTR *topPtr)
    Retirar (pop) un valor de la pila.
int isEmpty(STACKNODEPTR topPtr)
    Determinar si la pila está vacía.
void printStack(STACKNODEPTR topPtr)
    Imprimir la pila.
```

12.14 Modifique el programa evaluador postfijo del ejercicio 12.13, de tal forma que pueda procesar operandos de enteros mayores que 9.

12.15 (*Simulación de supermercado*). Escriba un programa que simule una línea de caja en el supermercado. La línea es una cola de espera. Los clientes llegan a intervalos enteros al azar, de 1 a 4 minutos. También, cada cliente es atendido en intervalos enteros al azar, de entre 1 y 4 minutos. Obviamente, las velocidades necesitan equilibrarse. Si la velocidad de llegada promedio es mayor que la velocidad promedio de servicio, la cola crecerá en forma indefinida. Inclusive tratándose de velocidades equilibradas, el azar todavía puede causar líneas largas. Ejecute la simulación de supermercado para un día de 12 horas (720 minutos) utilizando el algoritmo siguiente:

- 1) Escoja un entero al azar entre 1 y 4 para determinar el minuto en el cual el primer cliente llega.
- 2) En el momento de llegada del primer cliente:
Determine el tiempo de servicio para ese cliente (entero al azar de 1 a 4);
Empiece a darle servicio al cliente;

programe el tiempo de llegada del siguiente cliente (un entero al azar de 1 a 4, añadido al tiempo actual).

3) Para cada minuto del día:

Si llega el siguiente cliente,

Póngalo en la cola;

Programe el tiempo de llegada del siguiente cliente;

Si para el último cliente el servicio ya fue terminado;

Dígalo así

Retire de la cola para darle servicio al cliente siguiente

Determine el tiempo de terminación del servicio del cliente (entero al azar de 1 a 4, añadido al tiempo actual).

Ahora ejecute su simulación durante 720 minutos y conteste cada una de las siguientes preguntas:

- ¿Cuál es en cualquier momento el número máximo de clientes en la cola?
- ¿Cuál es la espera más larga experimentada por un cliente?
- ¿Qué pasa si el intervalo de llegadas es modificado de 1 a 4 minutos, a 1 a 3 minutos?

12.16 Modifique el programa de la figura 12.19 para permitir que el árbol binario contenga valores duplicados.

12.17 Escriba un programa basado en el programa de la figura 12.19 que introduzca una línea de texto, divida léxicamente la oración en palabras por separado, inserte las palabras en un árbol de búsqueda binaria, e imprima los recorridos en orden, preorden y postorden del árbol.

Sugerencia: lea la línea de texto a un arreglo. Utilice `strtok` para dividir léxicamente el texto. Una vez encontrada una división, genere un nuevo nodo en el árbol, asigne el apuntador que regresa `strtok` al miembro `string` del nuevo nodo, e inserte el nodo en el árbol.

12.18 En este capítulo, vimos al crear un árbol de búsqueda binario, que la eliminación de duplicados es sencilla. Describa cómo ejecutaría usted la eliminación de duplicados utilizando únicamente un arreglo de un solo subíndice. Compare el rendimiento de la eliminación de duplicados basado en arreglos, con el rendimiento de la eliminación de duplicados basado en el árbol de búsqueda binario.

12.19 Escriba una función `depth` que recibe un árbol binario y que determine cuántos niveles tiene.

12.20 (*Impresión recursiva de una lista en orden inverso*). Escriba una función `printListBackwards` que en forma recursiva extraiga los elementos en una lista en orden inverso. Utilice su función en un programa de prueba que origine una lista ordenada de enteros e imprima la lista en orden inverso.

12.21 (*Búsqueda de una lista en forma recursiva*). Escriba una función `searchList` que busque en una lista enlazada en forma recursiva, buscando un valor específico. Si éste es hallado la función deberá regresar un apuntador al valor; de lo contrario, `NULL` deberá ser regresado. Utilice su función en un programa de prueba que origine una lista de enteros. El programa deberá solicitar al usuario un valor a localizar dentro de la lista.

12.22 (*Borrado de árbol binario*). En este ejercicio, analizamos el borrado de elementos de árboles de búsqueda binarios. El algoritmo de borrado no es tan sencillo como el algoritmo de inserción. Al borrar un elemento se presentan tres casos distintos —el elemento está contenido en un nodo de hoja (es decir, no tiene hijo), el elemento está contenido en un nodo que tiene un hijo, o el elemento está contenido en un nodo que tiene dos hijos.

Si el elemento a borrarse está contenido en un nodo de hoja, el nodo se borra y el apuntador en el nodo padre se define a `NULL`.

Si el elemento a borrarse está contenido en un nodo con un hijo, el apuntador en el nodo padre se define para apuntar al nodo hijo y el nodo conteniendo el elemento de dato se borra. Esto hace que el nodo hijo tome el lugar del nodo borrado dentro del árbol.

El último caso es el más difícil. Cuando es borrado un nodo que tiene dos hijos, otro nodo dentro del árbol debe tomar su lugar. Sin embargo, el apuntador en el nodo padre no puede simplemente asignarse para apuntar a uno de los hijos del nodo a borrarse. En la mayor parte de los casos, el árbol de búsqueda binario no cumpliría con la característica siguiente de los árboles de búsqueda binarios: *los valores en cualquier subárbol izquierdo son menores que el valor en el nodo padre, y los valores en cualquier subárbol derecho son mayores que el valor en el nodo padre*.

¿Qué nodo se utilizará como *nodo de remplazo* para poder mantener esta característica? Ya sea el nodo que contenga el valor más grande en el árbol menor que el valor del nodo a borrarse, o el nodo que contenga el valor más pequeño en el árbol mayor que el valor del nodo a borrarse. Consideremos el nodo con el valor menor. En un árbol de búsqueda binario, el valor menor que el valor del padre se localiza en el subárbol izquierdo del nodo padre y se garantiza que está contenido en el nodo más a la derecha del subárbol. Este nodo se localiza dirigiéndose hacia abajo por el subárbol izquierdo, hacia la derecha, hasta que resulte `NULL` el apuntador hacia el hijo derecho del nodo actual. Estamos ahora apuntando al nodo de remplazo, que es o un nodo de hoja o un nodo con un hijo a su izquierda. Si el nodo de remplazo es un nodo de hoja, los pasos para llevar a cabo el borrado son como sigue:

- 1) Almacene el apuntador al nodo a borrarse en una variable de apuntador temporal (este apuntador se utiliza para liberar memoria dinámicamente asignada).
- 2) Defina el apuntador en el padre del nodo a borrarse, para que apunte al nodo de remplazo.
- 3) Defina a `NULL` el apuntador en el padre del nodo de remplazo.
- 4) Defina el apuntador al subárbol derecho, en el nodo de remplazo, para que apunte al subárbol derecho del nodo a borrarse.
- 5) Borre el nodo al cual apunta la variable de apuntador temporal.

Los pasos de borrado para un nodo de remplazo con un hijo izquierdo son similares a los de un nodo de remplazo sin hijos, pero el algoritmo también debe mover el hijo a la posición del nodo de remplazo en el árbol. Si el nodo de remplazo es un nodo con un hijo izquierdo, los pasos a llevarse a cabo para el borrado son como sigue:

- 1) Almacene en una variable de apuntador temporal el apuntador al nodo a borrarse.
- 2) Defina el apuntador en el padre del nodo a borrarse, para apuntar al nodo de remplazo.
- 3) Defina el apuntador en el padre del nodo de remplazo, para apuntar al hijo izquierdo del nodo de remplazo.
- 4) Defina el apuntador al subárbol derecho en el nodo de remplazo, para apuntar al subárbol derecho del nodo a borrarse.
- 5) Borre el nodo hacia el cual apunta la variable de apuntador temporal.

Escriba la función `deleteNode`, que toma como sus argumentos un apuntador al nodo raíz del árbol y el valor a borrarse. La función deberá localizar el nodo que contenga el valor a borrarse en el árbol y utilizar los algoritmos analizados aquí para borrarlo. Si no se encuentra el valor en el árbol, la función deberá imprimir un mensaje, que indique si se borra o no el valor. Modifique el programa de la figura 12.19 para utilizar esta función. Después de borrar un elemento, llame a las funciones de recorrido `inOrder`, `preOrder` y `postOrder` para confirmar que la operación de borrado fue correctamente ejecutada.

12.23 (*Búsqueda de árbol binario*). Escriba la función `binaryTreeSearch`, que intenta localizar un valor especificado en un árbol de búsqueda binario. La función deberá tomar como argumentos un apuntador al nodo raíz del árbol binario y una clave de búsqueda a localizar. Si se encuentra el nodo que contenga la clave de búsqueda, la función deberá regresar un apuntador a dicho nodo; de lo contrario, la función deberá regresar un apuntador `NULL`.

12.24 (*Recorrido de árbol binario en orden de nivel*). El programa de la figura 12.19 ilustró tres métodos recursivos de recorrer un árbol binario —recorrido en orden, recorrido preorden y recorrido postorden. En este ejercicio se presenta el *recorrido en orden de nivel* de un árbol binario, en el cual los valores de nodo se imprimen nivel por nivel, iniciándose en el nivel del nodo raíz. En cada nivel los nodos se imprimen de

izquierda a derecha. El recorrido en orden de nivel no es un algoritmo recursivo. Utiliza la estructura de datos de cola de espera para controlar la salida de los nodos. El algoritmo es como sigue:

- 1) Inserte en la cola de espera el nodo raíz.
- 2) En tanto existan nodos en la cola,

Obtenga el nodo siguiente en la cola

Imprima el valor del nodo

Si no es `NULL` el apuntador al hijo izquierdo del nodo

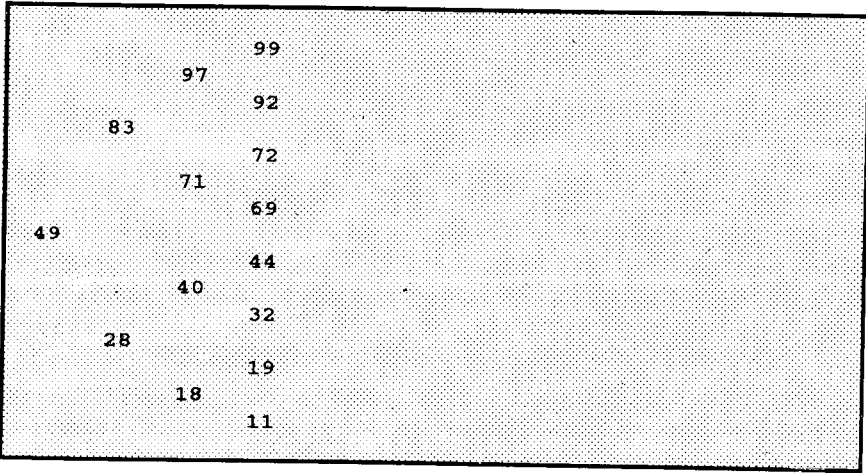
Inserte en la cola el nodo del hijo izquierdo

Si no es `NULL` el apuntador al hijo derecho del nodo

Inserte en la cola el nodo del hijo derecho.

Escriba la función `levelOrder` para ejecutar un recorrido en orden de nivel de un árbol binario. La función deberá tomar como argumento un apuntador al nodo raíz del árbol binario. Modifique el programa de la figura 12.19 para utilizar esa función. Compare la salida correspondiente a esa función con las salidas de otros algoritmos de recorrido, para ver si funcionó correctamente. (Nota: también será necesario modificar e incorporar dentro de este programa las funciones de procesamiento de colas de la figura 12.13.)

12.25 (Impresión de árboles). Escriba una función recursiva `outputTree` para desplegar en pantalla un árbol binario. La función deberá de extraer el árbol, renglón por renglón, con la parte superior del árbol en la parte izquierda de la pantalla, y la parte inferior del árbol hacia la derecha de la pantalla. Cada renglón es extraído en forma vertical. Por ejemplo, el árbol binario ilustrado en la figura 12.22 es extraído como sigue:



Note que el nodo de hoja más a la derecha aparece en la parte superior de la salida, en la columna más a la derecha y el nodo de raíz aparece a la izquierda de la salida. Cada columna extraída se inicia cinco espacios a la derecha de la columna anterior. La función `outputTree` debe recibir como argumentos un apuntador al nodo raíz del árbol y un entero `totalSpaces`, representando el número de espacios que anteceden al valor a ser extraído (esta variable deberá iniciarse en cero, de tal forma que el nodo raíz sea extraído en la parte izquierda de la pantalla). La función utiliza para sacar el árbol un recorrido enorden modificado —se inicia en el nodo más a la derecha en el árbol y funciona de regreso hacia el izquierdo. El algoritmo es como sigue:

- En tanto no sea `NULL` el apuntador al nodo actual
- Llame recursivamente `outputTree` con el subárbol derecho del nodo actual y `totalSpace + 5`
- Utilice una estructura `for` para contar de 1 hasta `totalSpace` y extraiga los espacios.

- Extraiga el valor del nodo actual
- Defina el apuntador al nodo actual para que apunte al subárbol izquierdo del nodo actual.
- Incrementa `totalSpaces` en 5.

Sección especial: Cómo construir su propio compilador

En los ejercicios 7.18 y 7.19 presentamos el lenguaje de máquina Simpletron (LMS) y creamos el simulador de computadora Simpletron para ejecutar programas escritos en LMS. En esta sección, construiremos un compilador que convierte a LMS programas escritos en un lenguaje de programación de alto nivel. Esta sección se “encadena” con todo el proceso de programación. Escribiremos programas en este nuevo lenguaje de alto nivel, compilaremos los programas en el compilador que construiremos, y ejecutaremos los programas en el simulador que construimos en el ejercicio 7.19.

12.26 (El Lenguaje Simple). Antes de que empecemos a construir el compilador, analizamos un lenguaje de alto nivel simple, aunque poderoso, similar a las versiones primeras del popular lenguaje BASIC. Llamamos el lenguaje *Simple*. Cada *enunciado Simple* consiste de un *número de línea* y de una *instrucción Simple*. Los números de línea deben aparecer en orden ascendente. Cada instrucción inicia con uno de los siguientes *comandos Simple*: `rem`, `input`, `let`, `print`, `goto`, `if/goto` o `end` (vea la figura 12.23). Todos los comandos, a excepción de `end`, pueden ser utilizados en forma repetida. Simple sólo evalúa expresiones enteras utilizando los operadores `+`, `-`, `*`, y `/`. Estos operadores tienen la misma precedencia que en C. Pueden utilizarse paréntesis para cambiar el orden de evaluación de una expresión.

Nuestro compilador Simple reconoce únicamente letras minúsculas. Todos los caracteres en un archivo Simple deberán ser minúsculas (las letras mayúsculas resultarán en un error de sintaxis, a menos de que aparezcan en un enunciado `rem`, en cuyo caso serán ignoradas). Un *nombre variable* es una sola letra. Simple no permite nombres descriptivos de variables, por lo que las variables deberán ser explicadas en comentarios, para documentar su uso dentro del programa. Simple utiliza únicamente variables enteras. Simple no tiene declaraciones de variables —el mero mencionar de un nombre de variable en un programa hace que la variable sea declarada y automáticamente inicializada a cero. La sintaxis de Simple no permite

Comando	Enunciado de muestra	Descripción
rem	50 rem this is a remark	Cualquier texto que siga al comando <code>rem</code> es sólo para fines de documentación y será ignorado por el compilador
input	30 input x	Despliega un signo de interrogación, para solicitar al usuario que introduzca un entero. Lee dicho entero del teclado y almacena el entero en <code>x</code> .
let	80 let u = 4 * (j - 56)	Asigna <code>u</code> el valor de <code>4 * (j - 56)</code> . Note que a la derecha del signo igual puede aparecer una expresión arbitrariamente compleja.
print	10 print w	Despliega el valor de <code>w</code> .
goto	70 goto 45	Transfiere el control del programa a la línea 45.
if/goto	35 if i == z goto 80	Compara <code>i</code> con <code>z</code> buscando igualdad y transfiere el control del programa a la línea 80, si la condición es verdadera; de lo contrario, continúa la ejecución en el enunciado siguiente.
end	99 end	Termina la ejecución del programa.

Fig. 12.23 Comandos Simple.

la manipulación de cadenas (lectura de una cadena, escritura de una cadena, comparación de cadenas, etcétera).

Si en un programa Simple se encuentra una cadena (después de un comando distinto que `rem`), el compilador generará un error de sintaxis. Nuestro compilador supondrá que los programas Simple se escriben en forma correcta. En el ejercicio 12.19 se le solicita al estudiante que modifique el compilador para que lleve a cabo verificación de errores de sintaxis.

Simple utiliza durante la ejecución del programa el enunciado condicional `if/goto` y el enunciado incondicional `goto` para alterar el flujo de control. Si la condición en el enunciado `if/goto` es verdadera, el control se transfiere a una línea específica dentro del programa. Los siguientes operadores de igualdad relacionales son válidos en un enunciado `if/goto`: `=`, `<`, `>`, `<=`, `>=`, `==`, o `!=`. La precedencia de estos operadores es la misma que en C.

Consideremos ahora varios programas Simple que demuestran las características de Simple. El primer programa (la figura 12.24) lee dos enteros del teclado, almacena los valores en las variables `a` y `b` y calcula e imprime su suma (almacenada en la variable `c`).

El programa de la figura 12.25 determina e imprime cuál es el mayor de dos enteros. Los enteros son introducidos desde el teclado y almacenados en `s` y `t`. El enunciado `if/goto` prueba la condición `s > t`. Si la condición es verdadera, el control se transfiere a la línea 90 y se extrae `s`; de lo contrario, se extrae `t` y el control es transferido al enunciado `end` en la línea 99, donde el programa termina.

```
10 rem  determine and print the sum of two integers
15 rem
20 rem  input the two integers
30 input a
40 input b
45 rem
50 rem  add integers and store result in c
60 let c = a + b
65 rem
70 rem  print the result
80 print c
90 rem  terminate program execution
99 end
```

Fig. 12.24 Cómo determinar la suma de dos enteros.

```
10 rem  'determine the larger of two integers
20 input s
30 input t
32 rem
35 rem  test if s >= t
40 if s >= t goto 90
45 rem
50 rem  t is greater than s, so print t
60 print t
70 goto 99
75 rem
80 rem  s is greater than or equal to t, so print s
90 print s
99 end
```

Fig. 12.25 Cómo encontrar el mayor de dos enteros.

Simple no posee una estructura de repetición (como `for`, `while` o `do/while` de C). Sin embargo, Simple puede simular cada una de las estructuras de repetición de C, mediante el uso de los enunciados `if/goto` y `goto`. La figura 12.26 utiliza un ciclo controlado por centinela para calcular los cuadrados de varios enteros. Cada entero es introducido desde el teclado y almacenado en la variable `j`. Si el valor introducido es el centinela `-9999`, el control se transfiere a la línea 99, donde el programa termina. De lo contrario, `k` es asignado el valor del cuadrado de `j`, `k` es extraído a la pantalla y el control se pasa a la línea 20, donde se introduce el siguiente entero.

Utilizando los programas de muestra de la figura 12.24, figura 12.25 y figura 12.26 como guía, escriba un programa Simple para llevar a cabo cada uno de los siguientes:

- a) Introducir tres enteros, determinar su promedio e imprimir el resultado.
- b) Utilizar un ciclo controlado por centinela para introducir 10 enteros y calcular e imprimir su suma.
- c) Utilizar un ciclo controlado por contador para introducir 7 enteros, algunos positivos y algunos negativos, y calcular e imprimir su promedio.
- d) Introducir una serie de enteros y determinar e imprimir cuál es el mayor. El primer entero introducido indicará cuántos números deberán de ser procesados.
- e) Introducir 10 enteros e imprimir el más pequeño.
- f) Calcular e imprimir la suma de enteros pares, desde 2 hasta 30.
- g) Calcular e imprimir el producto de los enteros impares desde 1 hasta 9.

12.27 (Cómo construir un compilador; *prerrequisito: haber terminado los ejercicios 7.18, 7.19, 12.12, 12.13 y 12.26*). Ahora que se ha presentado el lenguaje Simple (ejercicio 12.26), analizamos cómo construir nuestro compilador Simple. Primero, consideraremos el proceso mediante el cual un programa Simple se convierte a LMS y es ejecutado por el simulador Simpletron (vea la figura 12.27). Un archivo que contiene un programa Simple es leído por el compilador y convertido a código LMS. El código LMS es extraído a un archivo en disco, en el cual las instrucciones LMS aparecen, una por línea. A continuación el archivo LMS es cargado en el simulador Simpletron, y los resultados se envían a un archivo en disco y a pantalla. Note que el programa Simpletron desarrollado en el ejercicio 7.19, tomó su entrada desde el teclado. Deberá ser modificado para poder leerse desde un archivo, de tal forma que pueda ejecutar los programas producidos por nuestro compilador.

Para convertirlo a LMS, el compilador ejecuta dos *pasadas* del programa Simple. La primera pasada construye una *tabla simbólica*, en la cual se almacenan todos los *números de línea*, *nombres de variable* y *constantes* del programa Simple, con su tipo y posición correspondiente en el código final LMS (la tabla simbólica se analiza en detalle más adelante). Esta primera pasada también produce las instrucciones correspondientes LMS para cada enunciado Simple. Como veremos, si el programa Simple contiene

```
10 rem  calculate the squares of several integers
20 input j
23 rem
25 rem  test for sentinel value
30 if j == -9999 goto 99
33 rem
35 rem  calculate square of j and assign result to k
40 let k = j * j
50 print k
53 rem
55 rem  loop to get next j
60 goto 20
99 end
```

Fig. 12.26 Cómo calcular los cuadrados de varios enteros.

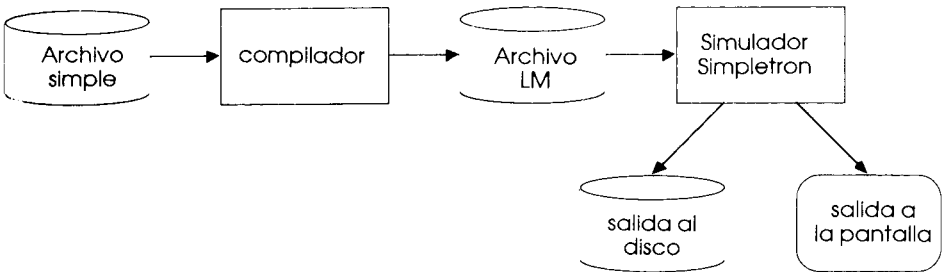


Fig. 12.27 Cómo escribir, compilar y ejecutar un programa de lenguaje Simple.

enunciados en el programa que transfieren el control a una línea más adelante, la primera pasada resulta en un programa LMS que contiene algunas instrucciones incompletas. La segunda pasada del compilador localiza y completa las instrucciones incompletas, y extrae el programa LMS a un archivo.

Primera pasada

El compilador empieza leyendo a la memoria un enunciado del programa Simple. La línea deberá ser dividida en sus *componentes léxicos* individuales (es decir, las “piezas” de un enunciado) para proceso y compilación (para facilitar esta tarea se puede utilizar la función estándar de biblioteca `strtok`). Recuerde que cada enunciado se inicia con un número de línea seguido por un comando. Conforme el compilador divide un enunciado en *componentes léxicos*, si el componente es un número de línea, una variable o una constante, es colocado en la tabla simbólica. Un número de línea se coloca en la tabla simbólica sólo si se trata del primer componente de un enunciado. El `symbolTable` es un arreglo de estructuras `tableEntry` que representan cada uno de los símbolos en el programa. No existen restricciones en el número de símbolos que pueden aparecer en el programa. Por lo tanto, el `symbolTable` para un programa en particular pudiera resultar extenso. Por ahora, haga `symbolTable` un arreglo de 100 elementos. Una vez que el programa esté trabajando podrá aumentar o disminuir su tamaño.

La definición de estructura `tableEntry` es como sigue:

```
struct tableEntry {
    int symbol;
    char type; /* 'C', 'L', or 'V' */
    int location; /* 00 to 99 */
}
```

Cada estructura `tableEntry` contiene tres miembros. El miembro `symbol` es un entero que contiene la representación ASCII de una variable (recuerde que los nombres de variable son un carácter solo), un número de línea o una constante. El miembro `type` es uno de los siguientes caracteres, indicando el tipo del símbolo: ‘C’ para constante, ‘L’ para número de línea, o ‘V’ para variable. El miembro `location` contiene la posición en memoria Simpletron (00 a 99) al cual se refiere el símbolo. La memoria Simpletron es un arreglo de 100 enteros, en el cual se almacenan las instrucciones LMS y los datos. Para un número de línea, la posición es el elemento en el arreglo de memoria Simpletron en el cual las instrucciones LMS empiezan para el enunciado Simple. Para una variable o una constante, la posición es el elemento en el arreglo de memoria Simpletron en el cual está almacenada la variable o la constante. Las variables y las constantes están asignadas hacia atrás a partir del final de la memoria Simpletron. La primera variable o constante se almacena en la posición en 99, la siguiente en la posición en 98, etcétera.

La tabla simbólica juega una parte integral en la conversión de programa Simple a LMS. Aprendimos en el capítulo 7 que una instrucción LMS es un entero de cuatro dígitos, formados de dos partes —el *código de operación* y el *operando*. El código de operación está determinado en Simple mediante comandos. Por ejemplo, el comando sencillo `input` corresponde al código de operación LMS 10 (leer), y el comando Simple `print` corresponde al código de operación LMS 11 (escribir). El operando es una posición de

memoria, que contiene los datos sobre los cuales el código de operación ejecutará su tarea (por ejemplo, el código de operación 10 lee un valor del teclado y lo almacena en la posición de memoria especificado por el operando). El compilador busca en `symbolTable` para determinar la posición en la memoria Simpletron de cada símbolo, a fin de que la posición correspondiente pueda ser utilizada para completar las instrucciones LMS.

La compilación de cada enunciado Simple se basa en su comando. Por ejemplo, una vez inserto en la tabla simbólica el número de línea de un enunciado `rem`, el resto del enunciado es ignorado por el compilador, dado que el comentario es únicamente para fines de documentación. Los enunciados `input`, `print`, `goto` y `end` corresponden a las instrucciones `read`, `write`, `branch` (a una posición específica) y `halt` de LMS. Los enunciados que contengan estos comandos Simple se convierten directamente a LMS (note que un enunciado `goto` pudiera contener una referencia sin resolver, si la línea especificada se refiere a un enunciado posterior del archivo de programa Simple; esto a veces se conoce como una referencia hacia adelante).

Cuando se compila un enunciado `goto` con una referencia sin resolver, la instrucción LMS deberá ser marcada con una *bandera* para indicar que la instrucción será terminada en la segunda pasada del compilador. Las banderas se almacenan en un arreglo de 100 elementos `flags` del tipo `int`, en el cual cada elemento se inicializa a -1. Si la posición de memoria a la cual se refiere el número de línea en el programa Simple aún no es conocido (es decir, aún no aparece en la tabla simbólica), el número de línea se almacena en el arreglo `flags` en aquel elemento que tenga el mismo subíndice que la instrucción incompleta. El operando de la instrucción incompleta se establece temporalmente a 00. Por ejemplo, una instrucción incondicional de bifurcación (haciendo una referencia hacia adelante) se deja como +4000 hasta la siguiente pasada del compilador. La segunda pasada del compilador será descrita en breve.

La compilación de los enunciados `if/goto` y `let` es más complicada que la de otros enunciados —son los únicos enunciados que producen más de una instrucción LMS. Para un enunciado `if/goto`, el compilador produce código para probar la condición y si es necesario, para bifurcarse a otra línea. El resultado de la bifurcación podría ser una referencia sin resolver. Cada uno de los operadores relacionales y de igualdad pueden ser simulados utilizando las instrucciones `branch zero` y `branch negative` de LMS (o posiblemente una combinación de ambas).

En el caso de un enunciado `let`, el compilador produce un código para evaluar una expresión arbitrariamente compleja, compuesta de variables y/o constantes enteras. Las expresiones deberán de separar cada operando y operador con espacios. Los ejercicios 12.12 y 12.13 presentaron el algoritmo de conversión infijo a postfijo y el algoritmo de evaluación postfijo, utilizado por compiladores para evaluar expresiones. Antes de seguir adelante con su compilador, deberá de haber completado cada uno de esos ejercicios. Cuando un compilador se encuentra con una expresión, convierte la expresión de notación infija a notación postfija, y a continuación evalúa la expresión postfija.

¿Cómo es que el compilador produce el lenguaje máquina para evaluar una expresión que contenga variables? El algoritmo de evaluación postfijo contiene un “gancho” que permite que nuestro compilador genere instrucciones LMS, en vez de que proceda a evaluar la expresión. Para habilitar este “gancho” dentro del compilador, deberá ser modificado el algoritmo de evaluación postfijo, para que cada símbolo que encuentre (y posiblemente inserte) sea buscado en la tabla simbólica, determine la posición de memoria correspondiente a dicho símbolo, e *inserte (push) la posición de memoria sobre la pila en lugar del símbolo*. Cuando en la expresión postfija se encuentra un operador, son extraídas (pop) las dos posiciones de memoria de la parte superior de la pila, y se produce lenguaje máquina para llevar a cabo la operación, utilizando como operandos las posiciones de memoria. El resultado de cada una de estas subexpresiones se almacena en una posición de memoria temporal y se inserta (push) de vuelta a la pila, de tal forma que se pueda continuar con la evaluación de la expresión postfija. Cuando quede terminada la evaluación postfija, la posición de memoria que contiene el resultado es la única posición remanente sobre la pila. Esta es extraída (popped) y se generan instrucciones LMS para asignar el resultado a la variable en la parte izquierda del enunciado `let`.

segunda pasada

La segunda pasada del compilador lleva a cabo dos tareas; resolver cualquier referencia pendiente de resolver y extraer el código LMS a un archivo. La resolución de las referencias se ejecuta como sigue:

- 1) Busca en el arreglo `flags` alguna referencia sin resolver (es decir, un elemento que tenga un valor distinto a -1).
- 2) Localiza la estructura en el arreglo `symbolTable`, que contiene el símbolo almacenado en el arreglo `flags` (asegúrese que el tipo del símbolo es 'L', correspondiente a número de línea).
- 3) Inserte la posición de memoria del miembro de estructura `location` dentro de la instrucción con la referencia pendiente de resolver (recuerde que una instrucción que contenga una referencia pendiente de resolver tiene un operando 00).
- 4) Repita los pasos 1, 2 y 3, hasta que se llegue al final del arreglo `flags`.

Una vez terminado el proceso de resolución, todo el arreglo que contiene el código LMS es extraído a un archivo de disco, con una instrucción LMS por línea. Este archivo puede ser leído por el Simpletron para su ejecución (después de que el simulador haya sido modificado para que pueda leer su entrada a partir de un archivo).

Un ejemplo completo

El ejemplo siguiente ilustra una conversión completa de un programa Simple a LMS, como sería ejecutado por el compilador Simple. Considere un programa Simple, que introduce un entero y suma los valores desde 1 hasta dicho entero. El programa y las instrucciones LMS producidas por la primera pasada se ilustran en la figura 12.28. La tabla simbólica construida en la primera pasada se muestra en la figura 12.29.

La mayor parte de los enunciados Simple se convierten directamente a instrucciones sencillas LMS. Las excepciones a este programa son los comentarios, el enunciado `if/goto` de la línea 20, y los enunciados `let`. Los comentarios no se traducen a lenguaje máquina. Sin embargo, el número de línea correspondiente a un comentario es colocado en la tabla simbólica, por si en un enunciado `goto` o en un enunciado `if/goto` el número de línea aparece referenciado. La línea 20 del programa especifica que si la condición `y == x` es verdadera, el control del programa se transfiere a la línea 60. Dado que la línea 60 aparece más adelante en el programa, en la primera pasada del compilador no ha colocado aún 60 en la tabla simbólica (los números de línea se colocan en la tabla simbólica únicamente cuando aparecen como primera división léxica de un enunciado). Por lo tanto, en este momento no es posible determinar el operando de la instrucción `branch zero` LMS en la posición 03 en el arreglo de instrucciones LMS. El compilador colocará 60 en la posición 03 del arreglo `flags`, para indicar que esta instrucción se completará en la segunda pasada.

En el arreglo LMS debemos mantener registro de la posición de la siguiente instrucción, porque no existe una correspondencia uno a uno, entre los enunciados Simple y las instrucciones LMS. Por ejemplo, el enunciado `if/goto` de la línea 20 se compila en tres instrucciones LMS. Cada vez que se produce una instrucción, dentro del arreglo LMS debemos incrementar el *contador de instrucciones* a la siguiente posición. Note que el tamaño de la memoria Simpletron podría presentar un problema para los programas Simple que contengan muchos enunciados, variables y constantes. Es concebible que el compilador se quede sin memoria disponible. Para comprobar si éste es el caso, su programa deberá contener un *contador de datos*, a fin de llevar registro de la posición dentro del arreglo LMS en la cual se almacenará la siguiente variable o constante. Si el valor del contador de instrucciones es mayor que el valor que el del contador de datos, el arreglo LMS está lleno. En este caso, deberá darse por terminado el proceso de compilación y el compilador deberá imprimir un mensaje de error, indicando que durante la compilación se le terminó la memoria.

Revisión paso a paso del proceso de compilación

Recorramos el proceso de compilación del programa Simple de la figura 12.28. El compilador lee a la memoria la primera línea del programa

```
5 rem sum 1 to x
```

Programa simple	Localización e instrucción LMS		Descripción
5 rem sum 1 to x	ninguna		rem es ignorado
10 input x	00	+1099	leer x a la posición 99
15 rem check y == x	ninguna		rem es ignorado
20 if y == x goto 60	01	+2098	cargar y (98) a acumulador
	02	+3199	substrae x (99) del acumulador
	03	+4200	bifurcar cero a posición sin resolver
25 rem increment y	ninguna		rem es ignorado
30 let y = y + 1	04	+2098	cargar y al acumulador
	05	+3097	añadir 1 (97) al acumulador
	06	+2196	almacenar en posición temporal 96
	07	+2096	cargar de la posición temporal 96
35 rem add y to total	08	+2198	almacenar acumulador en y
	ninguna		rem es ignorado
40 let t = t + y	09	+2095	cargar t (95) al acumulador
	10	+3098	añadir y al acumulador
	11	+2194	almacenar en posición temporal 94
	12	+2094	cargar de la posición temporal 94
45 rem loop y	13	+2195	almacenar acumulador en t
	ninguna		rem es ignorado
50 goto 20	14	+4001	bifurcarse a la posición 01
55 rem output result	ninguna		rem es ignorado
60 print t	15	+1195	extraer t a pantalla
99 end	16	+4300	terminar la ejecución.

Fig. 12.28 Instrucciones LMS, producidas después de la primera pasada del compilador.

La primera división léxica del enunciado (el número de línea) se determina utilizando `strtok` (vea el capítulo 8 en el que se analizan las funciones de manipulación de las cadenas en C). El componente léxico regresado por `strtok` se convierte a un entero, mediante el uso de `atoi`, de tal forma que el símbolo 5 puede ser localizado en la tabla simbólica. Si no se encuentra el símbolo, se insertará en la tabla simbólica. Dado que estamos al principio del programa y ésta es la primera línea, en la tabla aún no existen símbolos. Por lo tanto, en la tabla simbólica se inserta 5 como del tipo L (número de línea) y se le asigna la primera posición en el arreglo LMS (00): aunque esta línea se trata de un comentario, aún así en la tabla simbólica se le asigna un espacio para el número de línea (por si es referenciado por un `goto` o por un `if/goto`). Como en relación con un enunciado `rem` no se genera ninguna instrucción LMS, no se incrementa el contador de instrucciones.

```
El enunciado
10 input x
```

es analizado léxicamente después. El número de línea 10 se coloca en la tabla simbólica como del tipo L y se le asigna la primera posición en el arreglo LMS (00 debido a que un comentario fue el que inició el